

STAGE 1 — PRODUCT DETAILS + COMMON KNOWLEDGE BASE

This is **only Stage 1** of the plan you requested.

For now, we are **not assigning responsibilities, features by member, milestones, or the day-by-day schedule**. This stage is only about making sure the entire team understands **what SONAR-SHIELD is, how it works, what each technical term means, and what every member needs to know before development starts**.

Your existing team document defines the project around SSS detection/segmentation, preprocessing, confidence/filtering, geotagging, API, UI and end-to-end integration.

1. PRODUCT OVERVIEW

Product name

SONAR-SHIELD

Full meaning

Shadow-Aware, Domain-Robust Marine Anomaly Intelligence System

The name describes the core idea:

- **Sonar** → the system works on Side-Scan Sonar imagery.
- **Shield** → the system is intended to reduce false positives and make detections more trustworthy.
- **Shadow-Aware** → acoustic shadow is used as supporting evidence.
- **Domain-Robust** → the system should be tested against changes in sonar appearance, noise, resolution and acquisition conditions.
- **Marine Anomaly Intelligence** → the output is not merely a bounding box; it is a detection plus supporting evidence, reliability, location and report.

2. WHAT PROBLEM ARE WE SOLVING?

A Side-Scan Sonar survey can generate large quantities of sonar imagery.

A human operator may need to inspect the imagery to find:

- debris
- derelict fishing gear
- pipes/cables
- shipwreck-related objects
- other unusual seabed targets

The difficult part is that sonar imagery does not behave like a normal RGB photograph.

A sonar image can contain:

strong acoustic returns

weak returns

acoustic shadows

seabed texture

noise

reverberation

natural structures

man-made structures

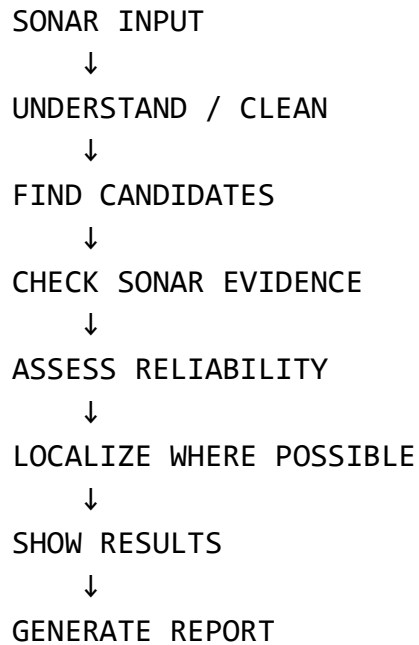
Therefore:

A bright sonar feature is not automatically debris, and a dark feature is not automatically empty space.

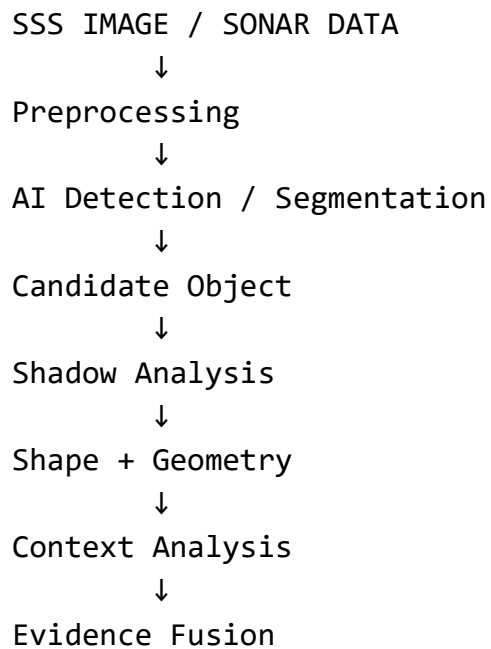
Your current product brief makes this explicit: the system needs an additional evidence layer after AI detection because sonar contains natural structures, shadows, noise and reverberation that can confuse a detector.

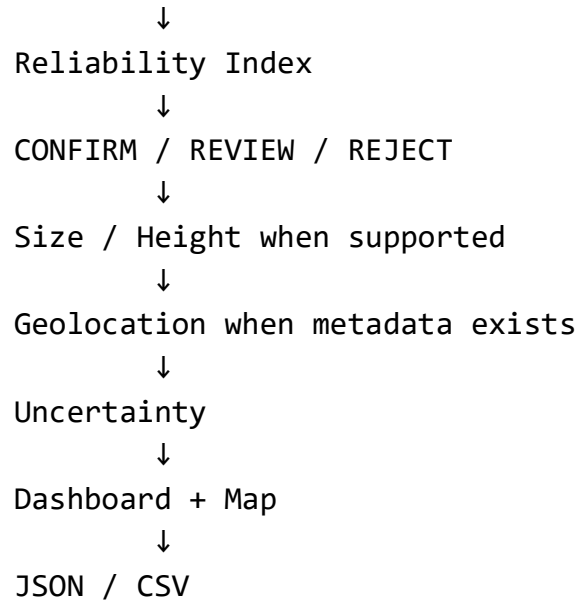
3. WHAT IS THE PRODUCT SUPPOSED TO DO?

At a high level:



More explicitly:





This is the product flow we have already defined in the development brief.

4. WHO WOULD USE THIS PRODUCT?

The immediate users are not ordinary consumers.

Think of:

Marine survey operators

Someone conducting an underwater survey wants to know:

"Where are the suspicious objects?"

Marine researchers

They may want:

"Which sonar regions contain structures worth investigating?"

Environmental monitoring teams

They may want:

"Where are possible derelict fishing objects or marine anomalies?"

AUV / USV operators

They may want:

"What did the vehicle detect during the survey?"

Human sonar analysts

They are especially important.

SONAR-SHIELD is **not supposed to replace the expert completely**.

One of the important product ideas is:

AI detects

↓

AI explains supporting evidence

↓

AI estimates reliability

↓

Human reviews uncertain cases

That is why the product has:

CONFIRM / REVIEW / REJECT

rather than pretending every AI result is perfect.

5. WHAT GOES INTO THE SYSTEM?

There are two broad categories.

A. Sonar imagery

The main AI input can be a side-scan sonar intensity image.

Typical representation:

grayscale / intensity image

The model sees numerical pixel values representing acoustic information.

B. Sonar/navigation metadata

Depending on the source data, metadata may contain things such as:

- GPS/location
- sonar range
- heading
- timestamp
- ping information
- vehicle position
- sensor offset
- navigation information

This information is **not the same thing as the image**.

The image tells us about the acoustic scene.

The metadata can help us understand:

where and under what acquisition geometry that image was produced.

6. WHAT COMES OUT?

For every candidate object, the system should ultimately be capable of producing something conceptually like:

```
{
  "object_id": "001",
  "class": "crab_pot",
  "detector_confidence": 0.87,
  "shadow_score": 0.76,
  "shape_score": 0.81,
  "context_score": 0.68,
  "reliability_index": 0.82,
  "decision": "CONFIRM",
  "estimated_length_m": 2.4,
  "estimated_width_m": 1.1,
  "estimated_height_m": null,
  "latitude": 10.52,
  "longitude": 76.21,
  "location_uncertainty_m": 5.2
}
```

Not every field will always be available.

For example:

No navigation metadata

↓

latitude = unavailable

longitude = unavailable

and:

Insufficient sonar geometry

↓

height = unavailable

That is a deliberate design choice.

7. THE PRODUCT HAS TWO LEVELS OF "CONFIDENCE"

This is one of the most important concepts everyone must understand.

Detector confidence

This comes from the AI model.

Example:

Crab pot
87%

That means approximately:

"The detector considers this visual pattern consistent with the crab-pot class."

It does **not** mean:

"There is an 87% scientifically calibrated probability that a physical crab pot exists."

Reliability Index

SONAR-SHIELD combines multiple pieces of evidence:

AI confidence
+
shadow evidence
+
shape
+
context
+
possibly persistence/geometry

and produces a **system-level reliability index**.

For example:

Detector confidence	0.87
Shadow evidence	0.76
Shape evidence	0.81
Context	0.68
	↓
Reliability Index	0.82

The product should use language such as:

Reliability Index: 0.82

rather than automatically calling that "82% probability."

This distinction is important for technical credibility.

8. WHAT EXACTLY IS THE AI MODEL DOING?

The AI model is primarily responsible for:

Candidate object detection.

It answers:

What class might this be?

Where is it?

How strong is the detector's prediction?

Example:

SSS image

↓

YOLO

↓

CRAB POT

confidence = 0.87

The model is therefore the **candidate generator**.

It is not the complete decision system.

9. WHY DON'T WE TRUST YOLO ALONE?

Because sonar contains confusing patterns.

Imagine:

ROCK

↓

YOLO

↓

"CRAB POT – 0.81"

The detector can be wrong.

Now SONAR-SHIELD asks:

Does the candidate have a plausible shadow?

Does its shape make sense?

Does its local seabed context make sense?

Is the detection persistent?

Is the geometry plausible?

Then the product can say:

Detector confidence: 0.81

Shadow evidence: weak

Shape evidence: weak

Context: weak

Reliability Index: 0.43

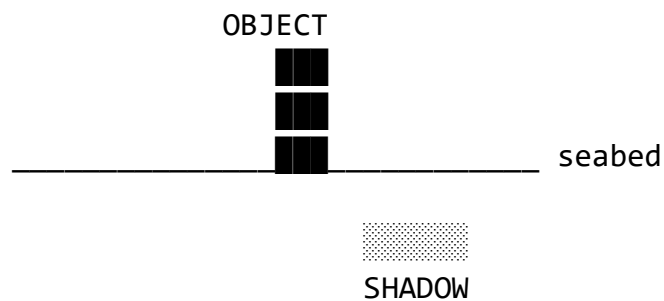
Decision: REVIEW / REJECT

That is the **central product philosophy**.

10. WHAT IS AN ACOUSTIC SHADOW?

This is fundamental knowledge for the entire team.

Suppose there is an object sticking above the seabed:



The acoustic return from the scene can contain a dark region behind the target.

That region is the **acoustic shadow**.

The shadow can tell us something about:

- object elevation
- geometry
- direction
- shape
- whether the bright return may correspond to a raised object

But:

Shadow alone is not proof of a man-made object.

Natural features can also produce confusing sonar returns.

Your current product brief explicitly treats shadow as **supporting evidence, not absolute proof**.

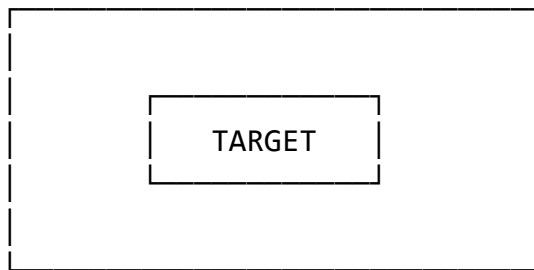
11. WHAT IS A BOUNDING BOX?

A bounding box is simply:

x1
y1
x2
y2

around a detected object.

Example:



YOLO's result may be:

```
{  
  "class": "crab_pot",  
  "confidence": 0.87,  
  "bbox": [120, 200, 300, 360]  
}
```

The product then uses that bounding box as the starting region for subsequent sonar analysis.

12. WHAT IS SEGMENTATION?

Detection says:

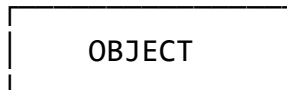
"The object is approximately here."

Segmentation says:

"These exact pixels belong to the object."

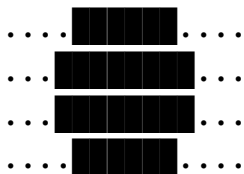
For example:

Detection:



versus:

Segmentation:



Your source material explicitly distinguishes YOLO-style detection from pixel-level segmentation and notes that segmentation requires pixel-labelled masks.

13. WHY DO WE HAVE BOTH DETECTION AND SEGMENTATION?

Because they answer different questions.

Detection

Useful when we need:

- object class
- approximate location
- bounding box
- confidence

Segmentation

Useful when we need:

- exact object shape
- pixel-level extent
- more precise geometric analysis
- masks for certain object categories

However:

We should not pretend every dataset supports segmentation.

If a dataset only has bounding boxes, it should not magically become a segmentation dataset.

14. WHAT IS A DATASET?

A dataset is simply the material we use to teach and test the model.

For detection:

IMAGE

+

BOUNDING BOX

+

CLASS

For segmentation:

IMAGE

+

PIXEL MASK

The team should think about datasets as having **different purposes**, not as one giant bucket.

For example:

Detection dataset

Segmentation dataset

Domain-diversity dataset

Negative dataset

Hard-negative dataset

Final test dataset

15. THE DATASET TYPES EVERYONE MUST UNDERSTAND

Training set

Used to teach the model.

TRAIN

The model is allowed to learn from it.

Validation set

Used during development.

VAL

Useful for:

- comparing model versions
- choosing thresholds
- tuning fusion parameters
- detecting overfitting

Final test set

Used for the final evaluation.

TEST

The team should **not use this to train the model**.

Error-mining pool

Used to discover mistakes:

MODEL

↓

ERROR-MINING SET

↓

FALSE POSITIVES

↓

HARD NEGATIVES

This is where the hard-negative examples should come from.

Negative benchmark

Contains images without target objects.

It helps answer:

"How often does the model hallucinate detections on background sonar?"

Your current plan already identifies the value of a separate negative test set for measuring whether hard-negative training actually reduces false detections.

16. WHAT IS A HARD NEGATIVE?

A hard negative is:

An image that contains no target, but looks sufficiently target-like that the model gets confused.

Examples:

rock
natural shadow
seabed texture
reverberation
bright return
noise

Example:

REAL IMAGE:
ROCK

MODEL:
CRAB POT 0.81 ❌

That rock image becomes useful training information.

The model is being taught:

"This thing looks confusing, but it is actually background."

17. WHAT IS A FALSE POSITIVE?

No actual object



Model predicts object

Example:

Actual: None

Prediction: Crab Pot

This is especially important for your project because false positives are one of the reasons sonar analysis is difficult.

18. WHAT IS A FALSE NEGATIVE?

Actual object exists



Model misses it

Example:

Actual: Crab Pot

Prediction: Nothing

This is **not** a hard negative.

It is a missed-target problem.

That means the team needs to distinguish:

False Positive

False Negative

True Positive

True Negative

19. WHAT IS DATA AUGMENTATION?

Data augmentation means creating controlled variations of training examples so the model sees more realistic variation.

Examples:

brightness variation
contrast variation
noise
blur
small geometric changes
intensity variation

In your project, augmentation may also simulate some sonar-specific disturbances.

The existing team plan explicitly mentions speckle-noise simulation, acoustic-shadow variation, geometric distortions and intensity variation as part of the training pipeline.

But everyone must understand:

An augmentation is useful only if it represents a plausible change in sonar appearance.

We don't blindly apply camera-style transformations.

20. WHAT IS PREPROCESSING?

Preprocessing prepares the sonar image before AI inference.

The current design includes things such as:

raw image
↓
denoising
↓
normalization

↓

CLAHE / local contrast enhancement

↓

processed image

The original team plan specifically identifies speckle-noise reduction, intensity normalization, CLAHE, shadow handling and dropout handling.

The goal is not:

"make the picture look pretty."

The goal is:

Improve consistency while preserving acoustic structures that matter.

21. WHAT IS CLAHE?

CLAHE stands for:

Contrast Limited Adaptive Histogram Equalization

In simple terms:

It improves local contrast instead of treating the entire image uniformly.

Why it can be useful:

One part of the sonar image
may be very dark

another part
may be much brighter

CLAHE can help reveal local structure.

But the team must experimentally verify that it improves detection.

22. WHAT IS DENOISING?

Denoising tries to remove unwanted noise while preserving meaningful structure.

Potential filters discussed in your project include:

- Lee
- Frost
- Non-Local Means
- median/other image-processing techniques

But again:

Don't assume stronger denoising is better.

Over-filtering could remove weak targets or alter shadows.

23. WHAT IS DOMAIN ROBUSTNESS?

This is another core idea that everyone should know.

Suppose the detector works on:

Sonar A

but fails on:

Sonar B

because of:

- different sonar instrument
- different frequency
- different resolution
- different substrate
- different range
- different acquisition conditions

Then the model isn't robust across domains.

So we want to evaluate:

TRAIN DOMAIN

↓

TEST DOMAIN

and ideally also:

different site

different survey

different instrument

This is why blindly mixing datasets is not enough.

24. WHAT IS DATA LEAKAGE?

Data leakage is when information from your evaluation data accidentally influences training.

Example:

Survey A

Frame 1 → TRAIN

Frame 2 → TEST

Frame 3 → TRAIN

If those images are nearly identical adjacent pings, your test score can look artificially good.

For sonar this is especially dangerous.

The current project brief already warns against putting very similar frames/patches from the same survey into train and test.

Everyone involved in datasets must understand this.

25. WHAT IS EVIDENCE FUSION?

Evidence fusion means:

Combining several signals instead of trusting one signal alone.

Our candidate may have:

AI confidence
Shadow evidence
Shape
Geometry
Context
Persistence

We combine these into one system-level assessment.

Conceptually:

AI
+
Shadow
+
Shape
+
Context
+
Geometry
↓
EVIDENCE VECTOR
↓
RELIABILITY INDEX

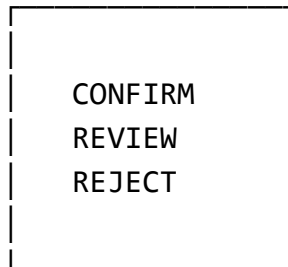
This is one of the main technical ideas of SONAR-SHIELD.

26. WHAT IS THE RELIABILITY ENGINE?

It turns evidence into a practical decision state.

Conceptually:

Reliability Index



Important:

These thresholds should be **validated using data**.

We shouldn't simply say:

0.80 = Confirm

0.50 = Review

and then call those scientifically established.

They are initially engineering thresholds that can be tuned.

27. WHAT IS "CONFIRM"?

It does **not** necessarily mean:

"The system has scientifically proven this is marine debris."

It means:

The evidence is sufficiently strong according to the system's validated decision criteria to mark this candidate as high-priority/confirmed within the prototype.

For human review, the interpretation should remain clear.

28. WHAT IS "REVIEW"?

Review means:

The AI sees something potentially important, but evidence is not strong enough for automatic confirmation.

This is an important product behavior.

Instead of:

AI sees weird thing

↓

FORCE CLASSIFICATION

we can say:

UNKNOWN / LOW-CONFIDENCE ANOMALY

↓

HUMAN REVIEW

29. WHAT IS UNKNOWN ANOMALY?

This is **not** a magic unknown-object detector.

At this stage, the idea is:

Known class confidence weak

OR

evidence inconsistent

OR

candidate doesn't fit trained categories

↓

UNKNOWN / REVIEW

The product therefore avoids forcing every unusual feature into an incorrect known class.

30. WHAT IS OBJECT SIZE ESTIMATION?

The detector gives dimensions in pixels.

Example:

width = 100 px

height = 40 px

To turn that into metres, we need a physical scale from the sonar geometry/range.

Conceptually:

pixel measurement

↓

sonar calibration/range

↓

physical distance

Output:

Estimated length: 2.4 m

Estimated width: 1.1 m

But:

Estimated is important.

If the necessary calibration/metadata isn't available, the system should say:

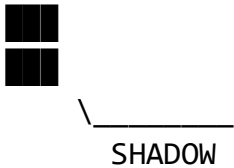
Size unavailable

31. WHAT IS HEIGHT ESTIMATION?

Height is different from length/width.

A potential approach is to use acoustic shadow geometry:

OBJECT



Shadow length combined with suitable sonar geometry can support an approximate height estimate.

But height requires more assumptions and metadata.

Therefore:

Height available?



YES → estimate + uncertainty

NO → unavailable

This should be treated as an **advanced conditional output**, not something the system must fabricate for every detection.

32. WHAT IS GEOLOCATION?

Geolocation converts sonar/image coordinates into geographic coordinates when navigation metadata supports it.

Conceptually:

pixel position



sonar-relative position

↓
navigation transform
↓
latitude / longitude

Potential metadata:

GPS
heading
range
timestamp
vehicle pose
sensor offset

33. WHAT IS GEOLOCATION UNCERTAINTY?

Even when we calculate a location, the value may not be exact.

Errors can come from:

- GPS
- sonar range
- image localization
- vehicle movement
- sensor offset
- heading
- geometry/calibration

Therefore:

Latitude: X
Longitude: Y
Uncertainty: $\pm Z$ m

is more realistic than pretending we know an exact point.

34. WHAT IS TRACKING/PERSISTENCE?

If the input consists of sequential sonar frames:

Frame 1 → candidate

Frame 2 → same candidate

Frame 3 → same candidate

then persistence can support the detection.

But this feature only makes sense when **sequential/overlapping data actually exists**.

If the input consists of unrelated still images:

image 1

image 2

image 3

we don't pretend that temporal tracking happened.

35. WHAT IS A REPORT?

The report converts AI results into structured information.

Example:

Object ID

Class

Detector confidence

Reliability Index

Decision

Dimensions

Latitude

Longitude

Uncertainty
Timestamp

Two expected formats:

JSON

Machine-readable.

CSV

Easy to inspect and import into other tools.

36. WHAT IS GIS?

GIS means:

Geographic Information System

In our application, it means displaying detections geographically.

Conceptually:

Detection



Lat/Lon



Map



Marker

The map is not just decoration.

It makes the output operationally useful:

"Where exactly was this anomaly found?"

37. WHAT IS THE DASHBOARD?

The dashboard is the user-facing layer.

It should eventually show:

Input

Upload sonar imagery.

Image analysis

- original image
- processed image
- detection boxes
- segmentation masks where available

Evidence

- detector confidence
- shadow evidence
- shape evidence
- context
- reliability index

Decision

- confirm
- review
- reject
- unknown

Geospatial

- map
- location
- uncertainty

Reports

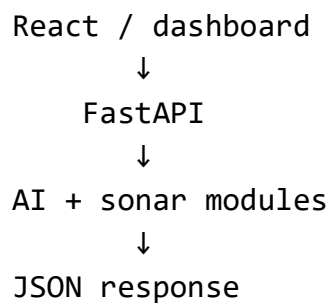
- JSON
- CSV

The original project plan similarly describes upload, detection overlays, confidence/class filtering and JSON/CSV report download for the dashboard.

38. WHAT IS FASTAPI?

FastAPI is the backend layer that exposes our processing pipeline as APIs.

Conceptually:



For example:

POST /detect
POST /analyze
POST /report

FastAPI is **not the AI**.

It is the bridge between:

frontend
and
backend processing

The existing team plan places a modular FastAPI backend with inference/report endpoints under the pipeline/backend side.

39. WHAT IS ONNX?

ONNX is a model interchange format.

Conceptually:

PyTorch-trained model



ONNX



different inference runtimes

It makes model deployment more portable.

40. WHAT IS TENSORRT?

TensorRT is an NVIDIA inference optimization/runtime technology.

It is used to optimize models for supported NVIDIA hardware.

Our thinking should be:

working model



ONNX



benchmark



TensorRT

Not:

TensorRT first

And we only claim speed/latency improvements that we actually benchmark.

41. WHAT IS DOCKER?

Docker packages the application and its dependencies so the environment can be reproduced.

Instead of saying:

"It only works on Member 3's laptop."

we want:

`docker compose up`

to start the system.

Docker is therefore about **deployment consistency**, not AI accuracy.

42. WHAT IS SYNTHETIC DATA?

Synthetic data is artificially generated training data.

For this project, Member 4 may create underwater 3D scenes with things like:

- pipes
- cylinders
- debris
- net-like structures
- shipwreck-like objects

and create sonar-like training imagery.

But:

Synthetic data is not automatically equivalent to real sonar.

Therefore:

Synthetic data

→ training augmentation

while:

Real sonar

→ validation / final testing

is the safer design.

Your original team plan positions Blender synthetic data as an additional source to improve the model, especially for rare classes.

43. WHAT IS HARD-NEGATIVE LEARNING?

The team should understand the complete loop:

TRAIN MODEL



RUN MODEL



FIND FALSE POSITIVES



VERIFY THEM



HARD NEGATIVES



ADD TO TRAINING DATA



FINE-TUNE



RE-EVALUATE

This is an iterative improvement loop.

Not:

train once

↓

done

44. WHAT IS AN ABLATION STUDY?

This is important for your project and everyone should know the basic idea.

You don't just say:

"Our full system is better."

You remove components one at a time.

For example:

A – YOLO baseline

B – YOLO + preprocessing

C – YOLO + hard negatives

D – YOLO + evidence fusion

E – Full SONAR-SHIELD

Then compare:

- precision
- recall
- mAP
- false positives

This helps answer:

Which component actually contributed something?

45. WHAT IS FAILURE ANALYSIS?

Instead of only celebrating correct predictions, we inspect the wrong ones.

Example:

FP → rock

FP → shadow

FP → seabed texture

FN → small target

FN → weak target

Then ask:

Why did this happen?

This is extremely important because your novelty involves reducing false positives.

46. THE PRODUCT'S MOST IMPORTANT TECHNICAL IDEA

The team should be able to explain this in one sentence:

SONAR-SHIELD does not simply detect suspicious objects; it uses sonar-specific evidence to assess whether a detection is trustworthy and provides uncertainty-aware geospatial output where the data supports it.

That is the product.

47. WHAT THE PRODUCT IS NOT

This is equally important.

SONAR-SHIELD is **not**:

Not a general underwater camera system

Our primary modality is SSS imagery.

Not a perfect 3D reconstruction system

SSS does not automatically give complete 3D geometry.

Not an exact height measurement system for every object

Height depends on sonar geometry and metadata.

Not a universal ghost-net detector unless we have suitable labelled data

A crab-pot dataset is not automatically a ghost-net dataset.

Not an autonomous replacement for a marine expert

The review pathway matters.

Not a guarantee that every unknown anomaly is artificial

Unknown means:

unusual / insufficiently classified / review-worthy.

48. COMMON DATASET KNOWLEDGE THE TEAM MUST HAVE

Before anyone says:

"We're training on five datasets."

they must be able to answer:

1. What classes are present?
2. What annotations exist?
3. Are they bounding boxes or masks?
4. What sonar instrument/site/domain produced them?
5. What is the licence?
6. Are train/test splits provided?
7. Can this dataset legitimately be mixed with another?
8. Is it real sonar or synthetic?
9. Does it contain the exact class we claim?
10. Could there be survey/scene leakage?

Your current team plan explicitly expects dataset collection, organization, split support and research around sonar datasets.

49. BASIC KNOWLEDGE REQUIRED BY EVERY TEAM MEMBER

Now we get to the second half of this stage.

Everyone needs the **same common foundation** first.

COMMON KNOWLEDGE — ALL 6 MEMBERS

Every member should understand these concepts before the project starts:

Sonar

- What Side-Scan Sonar is
- Acoustic intensity
- Acoustic shadow
- Seabed
- Range

- Ping
- Resolution
- Noise/reverberation
- Navigation metadata

AI

- What a model is
- Training
- Validation
- Testing
- Inference
- Detection
- Segmentation
- Confidence
- False positive
- False negative

Data

- Dataset
- Annotation
- Bounding box
- Mask
- Train/validation/test
- Data leakage
- Hard negative
- Data augmentation

System

- Frontend
- Backend
- API
- JSON
- Model
- preprocessing pipeline

- database/report
- GIS/map

Evaluation

- Precision
- Recall
- F1
- mAP
- IoU
- inference time
- false-positive rate

Product logic

Everyone should understand:

```
detect
↓
validate evidence
↓
reliability
↓
location
↓
report
```

This is important because even your non-core members may be questioned about the complete system. The original team plan explicitly notes the need for knowledge sharing across the team.

50. MEMBER 1 — KNOWLEDGE BASE

Member 1 is the AI-heavy person, so their knowledge depth needs to be much greater.

Must understand

Python

- variables
- functions
- classes
- files
- dictionaries
- JSON
- virtual environments
- packages

NumPy

- arrays
- dimensions
- slicing
- normalization
- numerical operations

PyTorch

- tensors
- model
- dataset
- dataloader
- training loop
- loss
- optimizer
- checkpoint

YOLO

- object detection
- bounding boxes
- class IDs

- confidence
- IoU
- NMS
- epochs
- batch size
- learning rate
- validation

Segmentation

- masks
- semantic segmentation
- IoU
- Dice
- pixel-level labels

Machine learning

- overfitting
- underfitting
- generalization
- train/validation/test
- data leakage
- class imbalance
- augmentation

Evaluation

Member 1 should be able to explain:

"Why did mAP change?"

not just:

"The mAP is 0.81."

51. MEMBER 2 — KNOWLEDGE BASE

Member 2 needs to understand the sonar itself more deeply.

Image processing

- grayscale/intensity
- normalization
- histogram
- contrast
- filtering
- denoising
- morphology
- contours
- connected components

Sonar concepts

- acoustic shadow
- range
- sonar geometry
- backscatter
- seabed texture
- resolution
- speckle/noise
- dropouts

Evidence analysis

- ROI
- shape
- aspect ratio
- local statistics
- shadow length
- shadow direction

- context

Reliability concepts

- feature
- score
- threshold
- fusion
- calibration
- uncertainty

Geospatial concepts

- latitude
- longitude
- coordinate system
- CRS
- transformation
- uncertainty
- sensor offset

Member 2 does not need to become a geophysicist in four days, but they must understand the assumptions behind every geometric calculation they implement.

52. MEMBER 3 — KNOWLEDGE BASE

Member 3 is the software architect.

Backend

- FastAPI
- routes
- request
- response

- JSON
- file upload
- validation
- exceptions

Software engineering

- modules
- interfaces
- separation of concerns
- logging
- testing
- configuration
- environment variables

Git

- clone
- branch
- commit
- pull
- merge
- pull request
- conflict resolution

Integration

Member 3 must understand:

```
model interface
+
sonar interface
+
report interface
+
frontend interface
```

Deployment

- Dockerfile
- image
- container
- volume
- environment
- Docker Compose

They don't need to become a DevOps engineer; they need enough knowledge to reliably package the project.

53. MEMBER 4 — KNOWLEDGE BASE

Member 4 needs enough technical understanding to correctly display the system.

Dashboard

- UI components
- image upload
- result display
- API calls
- JSON parsing
- status/error states

Map

- coordinates
- markers
- uncertainty
- missing-location state

AI result display

They must understand:

detector confidence
≠
reliability index

Otherwise the UI could misrepresent the science.

Synthetic data

Basic:

- Blender scene
- object
- camera
- lighting
- rendering
- annotation

Again, the synthetic image should not be presented as real sonar.

54. MEMBER 5 — KNOWLEDGE BASE

Member 5 should understand:

Dataset literacy

- classes
- labels
- annotations
- masks
- bounding boxes
- dataset splits

- licences

Research literacy

- paper
- benchmark
- dataset
- baseline
- limitation
- contribution

Sonar basics

Enough to understand:

Why is this image relevant to marine anomaly detection?

Data integrity

They must never tell the team:

"This dataset contains ghost nets"

unless the source actually confirms it.

55. MEMBER 6 — KNOWLEDGE BASE

Member 6 needs to understand the technical narrative.

They should be capable of explaining:

Problem

↓

Why SSS is hard

↓

AI detector

↓

Sonar evidence



Reliability



Geolocation



Dashboard

They should know:

- what each model does
- what dataset supports it
- what the key novelty is
- what the limitations are
- what the evaluation metrics mean

Most importantly:

Documentation must describe what the team actually implemented, not what was merely planned.

56. EVERY MEMBER MUST KNOW THESE 15 TERMS

Before development starts, everyone should know these without Googling:

1. Side-Scan Sonar
2. Acoustic shadow
3. Backscatter
4. Bounding box
5. Segmentation mask
6. YOLO
7. Inference
8. Precision
9. Recall
10. mAP

- 11. False positive
- 12. Hard negative
- 13. Evidence fusion
- 14. Geolocation
- 15. Uncertainty

57. EVERY MEMBER SHOULD BE ABLE TO ANSWER THESE BASIC QUESTIONS

Q1. What does our AI actually see?

A sonar intensity image, not a normal RGB photograph.

Q2. What does YOLO do?

Detect candidate objects and return class, bounding box and detector confidence.

Q3. Why isn't YOLO enough?

Sonar contains confusing natural structures, shadows and noise.

Q4. Why do we analyze the shadow?

It can provide supporting evidence about the physical target geometry.

Q5. What is the reliability index?

A system-level evidence score combining detector output with sonar-specific evidence.

Q6. What happens when the system is unsure?

It can mark the candidate for review rather than forcing a known class.

Q7. Can every detection be geolocated?

No. Geographic localization depends on sufficient sonar/navigation metadata.

Q8. Can every detection have a height?

No. Height requires suitable sonar geometry and shadow information.

Q9. Why do we need hard negatives?

To reduce false positives by teaching the detector about difficult background patterns.

Q10. What is our final test set?

A held-out dataset that is not used to train or mine hard negatives.

58. THE PRODUCT IN ONE EXAMPLE

Imagine the user uploads a sonar image.

Step 1

System preprocesses:

RAW

↓

normalized / enhanced

Step 2

YOLO says:

Potential Crab Pot
confidence = 0.84

Step 3

Evidence engine checks:

shadow = strong
shape = plausible
context = moderate

Step 4

Reliability engine calculates:

Reliability Index = 0.81

Step 5

System decides:

CONFIRM

Step 6

Metadata allows location calculation:

Latitude
Longitude
Uncertainty

Step 7

Dashboard shows:

[SONAR IMAGE]

Detected Object
Crab Pot

Detector Confidence
84%

Shadow Evidence

76%

Shape Evidence

81%

Reliability Index

81%

Decision

CONFIRM

Location

Available

Estimated Size

2.4m × 1.1m

Step 8

User downloads:

detection_report.json

detection_report.csv

That is the product experience we are trying to create.

59. PRODUCT PRINCIPLE WE SHOULD NEVER BREAK

The product should always prefer:

"I don't have enough evidence."

over:

"I'll confidently invent an answer."

That applies to:

- object class
- height
- location
- dimensions
- unknown anomalies

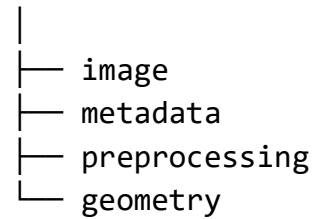
This is especially important in a sonar environment.

60. THE FINAL PRODUCT MODEL

Think of SONAR-SHIELD as **five layers**:

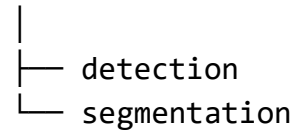
LAYER 1

SONAR UNDERSTANDING



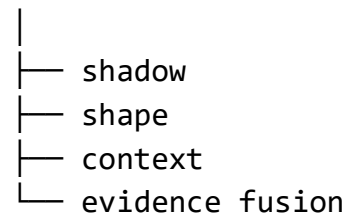
LAYER 2

AI



LAYER 3

SONAR INTELLIGENCE



LAYER 4
DECISION

- |
- |— reliability
- |— confirm
- |— review
- |— unknown

LAYER 5
ACTIONABLE OUTPUT

- |
- |— geolocation
- |— uncertainty
- |— map
- |— reports

That hierarchy should be understood by the entire team.

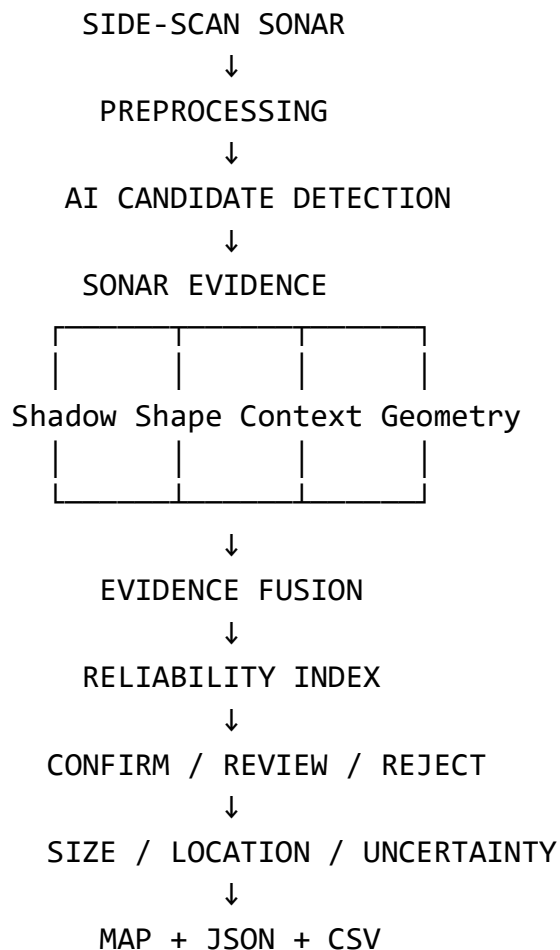
61. WHAT EVERYONE SHOULD UNDERSTAND BEFORE WE MOVE TO STAGE 2

At the end of this knowledge phase, nobody should be confused about:

- What is sonar?
- What is a sonar image?
- What is an acoustic shadow?
- What does YOLO do?
- What does segmentation do?
- What is a dataset?
- What is a label?
- What is a hard negative?
- What is preprocessing?
- What is evidence fusion?

What is a reliability index?
What is an unknown anomaly?
What is geolocation?
Why can geolocation fail?
Why can height fail?
What is a final test set?
What is data leakage?
What is an API?
What is GIS?
What is ONNX?
What is TensorRT?
What is Docker?

And most importantly, the whole team should understand this single flow:



This matches the product architecture you've been building, while making the assumptions and limitations explicit.

STAGE 1 CHECKPOINT

This stage should be approved before we move forward.

After approval, **Stage 2** will be a completely separate detailed document containing only:

Member 1 → exact responsibilities and deliverables

Member 2 → exact responsibilities and deliverables

Member 3 → exact responsibilities and deliverables

Member 4 → exact responsibilities and deliverables

Member 5 → exact responsibilities and deliverables

Member 6 → exact responsibilities and deliverables

STAGE 2 — FINAL RESPONSIBILITIES OF EACH TEAM MEMBER

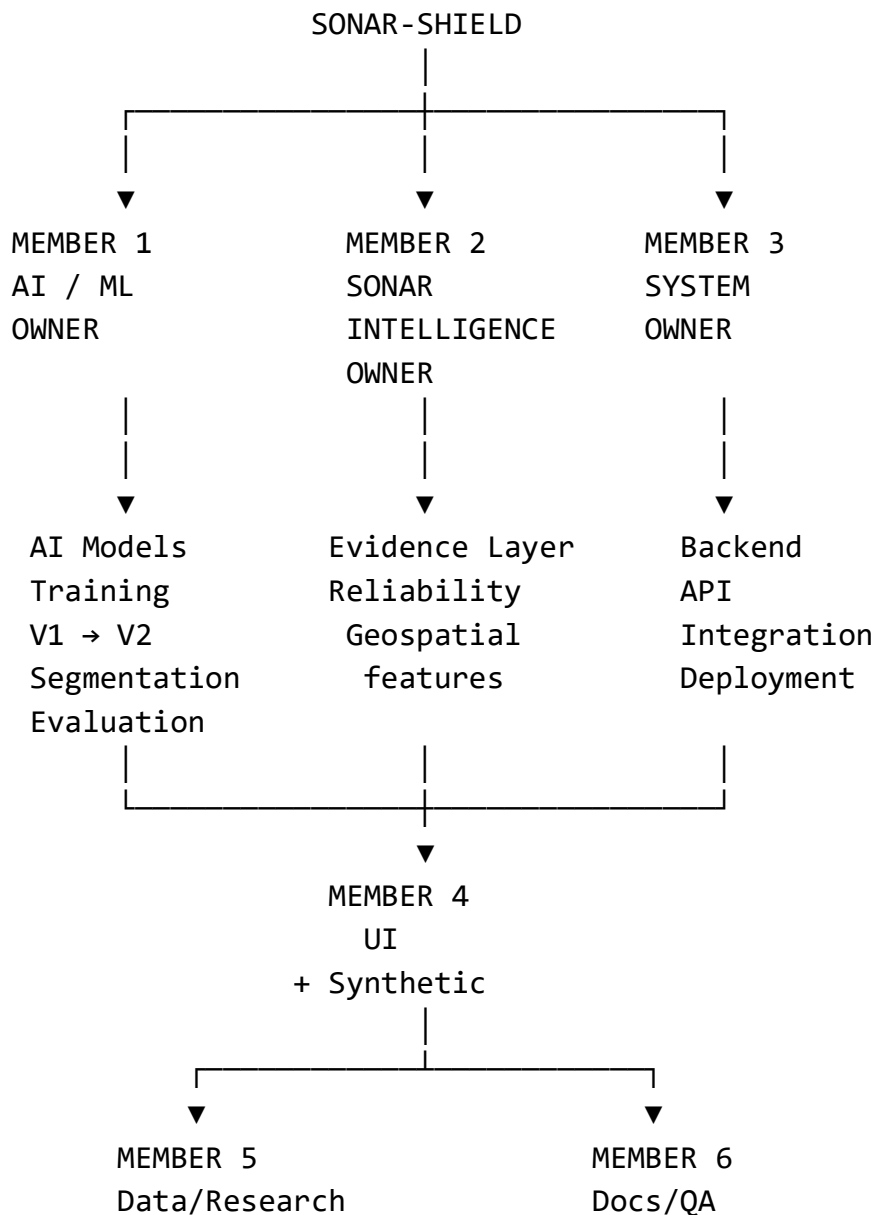
This stage defines **exactly who owns what**.

The important change from the original team document is that we are deliberately restructuring the work to fit your very short development window:

- **Member 1** now owns the **entire AI/ML training lifecycle**, including baseline training, error analysis, hard-negative generation/fine-tuning, segmentation, evaluation, and model export.
- **Member 2** is **not responsible for hard-negative training anymore**. They focus entirely on the **sonar-intelligence layer**: preprocessing, shadow/shape/context analysis, evidence fusion and reliability.
- **Member 3** owns the **software and deployment side**, including API, integration, tracking, geospatial transformation, reporting pipeline and Docker.
- **Member 4** owns UI and synthetic data.
- **Members 5 and 6** remain support roles.

This preserves the original document's heavy technical division around Members 1–3 while making the ownership much cleaner for your compressed schedule. The original brief assigns Member 1 the AI model, Member 2 preprocessing/confidence/geotagging, Member 3 integration, Member 4 UI/synthetic data, and Members 5–6 research/documentation.

1. TEAM STRUCTURE AT A GLANCE



The core principle is:

One person owns the implementation of one module.

Other people can help test or review it, but there is only **one accountable owner**.

2. MEMBER 1 — AI/ML LEAD

Role

AI/ML Model Owner

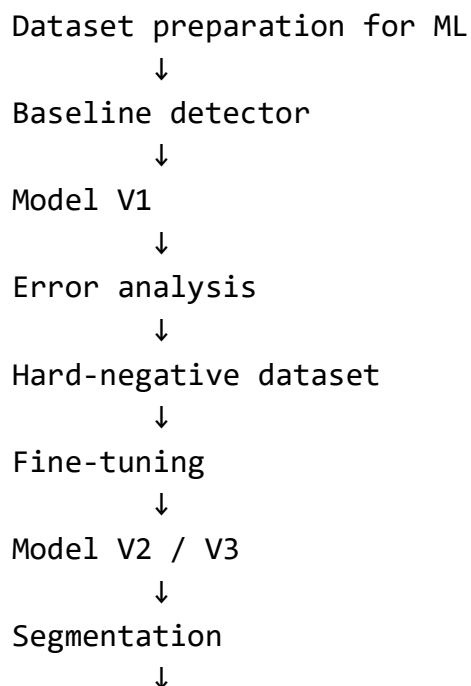
Member 1 owns **everything that teaches, evaluates and prepares the AI model.**

The original team brief already gives Member 1 responsibility for architecture selection, augmentation, training, difficult cases, evaluation, model export and experiment tracking.

We are expanding that ownership so the hard-negative loop also stays inside Member 1's AI pipeline.

2.1 Core ownership

Member 1 owns:



Evaluation



Final model selection



ONNX / TensorRT preparation

In simple words

Member 1 answers:

"How do we teach the AI to detect the target, improve its mistakes, and prove how well it works?"

3. MEMBER 1 — EXACT RESPONSIBILITIES

A. Dataset preparation for AI

Member 1 is responsible for taking the usable datasets and turning them into a training-ready format.

They must:

- inspect labels
- verify classes
- convert annotations when required
- create train/validation/test splits
- create the YOLO dataset configuration
- make sure classes match labels
- check for corrupted images
- detect duplicates/leakage
- maintain dataset versions

Deliverable

datasets/
└─ baseline/

```
|— images/  
|— labels/  
└— data.yaml
```

4. Member 1 — CLASS MANIFEST

Before training starts, Member 1 creates:

CLASS_MANIFEST.md

Example:

Class	Dataset	Annotation	Model task
Crab Pot	Ghost Pot	Bounding box	Detection
Maybe Crab Pot	Ghost Pot	Bounding box	Detection
Shipwreck	AI4Shipwrecks	Pixel mask	Segmentation
Pipeline	SubPipe	Detection label	Detection
Ghost Net	Must have verified labelled data	—	Only if available

This is extremely important.

Member 1 must **never add a class merely because it sounds useful**.

A class exists in the model only when there is appropriate labelled training data.

5. Member 1 — BASELINE MODEL

Member 1 owns the first model.

The process is:

Pretrained model



SSS dataset



Training



Validation



MODEL V1

The model must produce:

class

confidence

bounding box

Deliverables

models/

```
└─ v1/
    │  └─ model_v1.pt
    │  └─ classes.yaml
    └─ metrics.json
```

6. Member 1 — TRAINING EXPERIMENTS

Member 1 handles:

- model architecture
- image size
- epochs
- batch size
- learning rate experiments
- augmentation
- training configuration
- checkpoint management

But there is a rule:

Do not run ten random experiments without recording them.

Every experiment gets a version:

EXP-001

EXP-002

EXP-003

with:

model

dataset

parameters

metrics

observations

7. Member 1 — ERROR ANALYSIS

After Model V1 works, Member 1 runs it on an **error-mining pool**, not the locked final test set.

They identify:

False positives

rock → predicted target

shadow → predicted target

seabed → predicted target

noise → predicted target

False negatives

real target → missed

Other difficult cases

small target

weak return

partially obscured object
ambiguous target

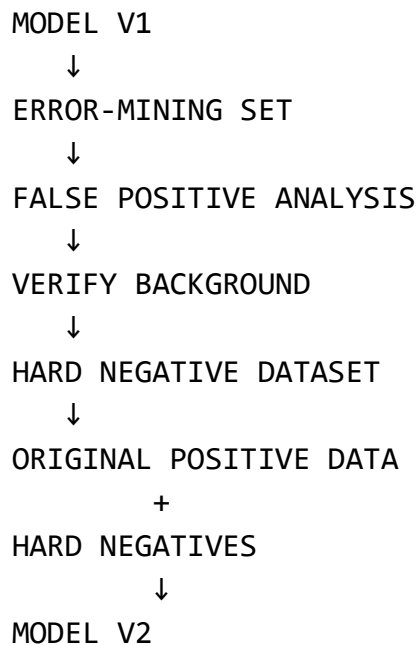
The original project plan explicitly calls for handling difficult cases such as small debris, ghost nets and objects hidden in acoustic shadows.

8. Member 1 — HARD-NEGATIVE LEARNING

This is the major change from the previous allocation.

Member 1 owns hard-negative learning.

Workflow:



Member 1 therefore owns both:

"Teach the model what the object looks like."

and:

"Teach the model what confusing non-objects look like."

This keeps the entire learning loop in one person's hands.

9. Member 1 — FALSE-NEGATIVE IMPROVEMENT

Hard negatives alone aren't enough.

Member 1 should separately collect difficult positives:

small target
weak target
partially visible target
low-contrast target

These become:

difficult positive examples

rather than hard negatives.

So Model V2 learns from:

Original positives
+
Difficult positives
+
Hard negatives

10. Member 1 — SEGMENTATION

Member 1 also owns segmentation.

However:

Detection and segmentation remain separate tasks.

For example:

YOLO

→ bounding box

Segmentation model

→ pixel mask

Member 1 must use a dataset that actually contains pixel-level masks for the segmentation task.

The original brief identifies semantic segmentation as part of the AI/ML responsibility.

11. Member 1 — DATA AUGMENTATION

Member 1 owns augmentation.

Potential experiments:

- intensity variation
- brightness/contrast variation
- noise
- blur
- mild geometric transformations
- sonar-like intensity changes
- shadow-related variations

The original team document specifically includes speckle noise simulation, acoustic-shadow variation, geometric distortion and intensity variation.

But Member 1 must test whether an augmentation **actually helps**.

12. Member 1 — EVALUATION

Member 1 owns the AI metrics.

They must produce:

Detection

- Precision
- Recall
- F1
- [mAP@0.5](#)
- [mAP@0.5:0.95](#)
- inference latency/FPS

Segmentation

- IoU
- Dice, if used
- representative masks

Robustness

- false positives
- false negatives
- performance by dataset/domain
- difficult-case analysis

13. Member 1 — ABLATION STUDY

Member 1 should own the model-side ablation experiment:

- A – Baseline YOLO
- B – + Preprocessing
- C – + Hard Negatives

D – + Evidence Fusion

E – Full system

The exact structure can be finalized once the actual pipeline exists.

The important thing is to determine:

Which component actually improves which metric?

14. Member 1 — FINAL MODEL SELECTION

Member 1 does NOT automatically choose the latest model.

For example:

V1:

Precision 0.80

Recall 0.86

V2:

Precision 0.87

Recall 0.82

The team must consider the application objective and overall validation/test behavior.

The final model is selected from evidence.

Output:

```
models/final/  
├─ final_model.pt  
├─ final_model.onnx  
├─ classes.yaml  
└─ metrics.json
```

15. Member 1 — MODEL EXPORT

Member 1 owns:

PyTorch

↓

ONNX

↓

TensorRT where practical

The original plan explicitly assigns ONNX/TensorRT export to the AI/ML role.

But the rule remains:

Deployment optimization happens after the model works.

16. MEMBER 1 — WHAT THEY HAND OFF

At the end of the AI phase, Member 1 gives Member 3:

```
final_model.pt  
final_model.onnx  
segmentation_model  
classes.yaml  
metrics.json  
inference.py
```

And gives Member 2:

```
model outputs  
bounding-box format  
class mapping  
confidence format
```

17. MEMBER 1 — WHAT THEY DO NOT OWN

Member 1 does **not** own:

- FastAPI
- frontend
- map rendering
- Docker
- UI design
- geospatial UI
- final application integration

They provide the model and its interface.

18. MEMBER 2 — SONAR INTELLIGENCE LEAD

Role

Sonar Evidence & Decision Intelligence Owner

Member 2 is now **completely removed from hard-negative training**.

This is intentional.

Their entire focus is:

"After the AI detects a candidate, how do we determine whether the sonar evidence supports that candidate?"

The original team brief gives Member 2 responsibility for preprocessing, confidence scoring, noise filtering and geotagging.

We refine that role into a focused sonar-intelligence module.

19. Member 2 — EXACT RESPONSIBILITIES

Member 2 owns:

Preprocessing

↓

Shadow Analysis

↓

Shape Analysis

↓

Geometry Features

↓

Context Analysis

↓

Evidence Fusion

↓

Reliability Index

↓

Confirm / Review / Reject

↓

Unknown/Review Path

And supporting sonar-derived physical features such as:

size

height where possible

Geographic transformation itself is moved to Member 3 so Member 2 isn't overloaded.

20. Member 2 — PREPROCESSING

Member 2 creates:

preprocessing/
└─ normalize.py

```
|— denoise.py
|— clahe.py
|— pipeline.py
```

Pipeline:

Raw SSS

↓

Normalization

↓

Denoising

↓

Contrast enhancement

↓

Output

Potential methods from the existing team plan include:

- Lee filtering
- Frost filtering
- Non-Local Means
- normalization
- CLAHE
- shadow/dropout handling

The original document explicitly lists these preprocessing directions.

21. Member 2 — PREPROCESSING EXPERIMENTS

Member 2 must compare:

RAW → MODEL

against:

PREPROCESSED → MODEL

rather than assuming preprocessing helps.

They should produce an experiment like:

Version	Processing	Precision	Recall	mAP
A	Raw			
B	Normalization			
C	Normalization + CLAHE			
D	Full			

22. Member 2 — SHADOW ANALYSIS

Member 2 owns the shadow engine.

Input:

```
processed sonar image
+
YOLO bounding box
```

Process:

```
Candidate
↓
Expanded ROI
↓
Acoustic intensity analysis
↓
Candidate dark region
↓
Shadow features
```

Possible output:

```
{
  "shadow_detected": true,
  "shadow_length_px": 83,
```

```
"shadow_direction": 1.2,  
"shadow_score": 0.76  
}
```

23. Member 2 — SHAPE ANALYSIS

Calculate:

- width
- height
- area
- aspect ratio
- perimeter
- orientation
- compactness
- elongation

The purpose is not to automatically classify objects.

It's to provide **supporting evidence**.

24. Member 2 — CONTEXT ANALYSIS

Member 2 analyzes the neighborhood around a candidate.

Potential features:

- local mean intensity
- local variance
- texture
- contrast
- nearby seabed characteristics

The question is:

Does the candidate meaningfully differ from its surrounding seabed environment?

25. Member 2 — GEOMETRY

Member 2 works with image/sonar geometry for:

- object/shadow relationship
- shadow-to-object ratio
- candidate orientation
- geometric plausibility
- size-related features

They do not need to make every geometric calculation a final physical measurement.

26. Member 2 — EVIDENCE FUSION

Member 2 receives:

AI confidence
+
Shadow evidence
+
Shape evidence
+
Context
+
Geometry

and creates:

Evidence Vector

Example:

```
{  
  "ai": 0.84,  
  "shadow": 0.76,  
  "shape": 0.81,  
  "context": 0.68  
}
```

27. Member 2 — RELIABILITY INDEX

Member 2 combines those values into the **SONAR-SHIELD Reliability Index**.

Important distinction:

Detector confidence

≠

Reliability Index

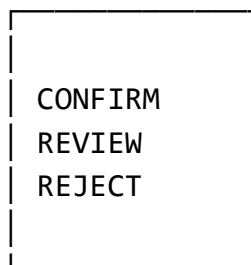
The Reliability Index is an engineering/decision score produced by the system.

It is not automatically a calibrated probability.

28. Member 2 — DECISION LOGIC

Member 2 defines:

Reliability



Thresholds should eventually be evaluated against validation examples.

29. Member 2 — UNKNOWN / REVIEW PATH

Member 2 handles situations where:

- detector confidence is weak
- evidence is contradictory
- geometry doesn't make sense
- the candidate doesn't clearly match trained classes

Output:

UNKNOWN / LOW-CONFIDENCE ANOMALY
HUMAN REVIEW REQUIRED

This is a **review pathway**, not a claim of full open-set recognition.

30. Member 2 — SIZE ESTIMATION

Member 2 owns sonar-derived dimensions.

Flow:

Bounding box

↓

Pixel dimensions

↓

Sonar range/scale

↓

Estimated physical dimensions

Output:

Estimated length
Estimated width

Only when sufficient calibration/metadata exists.

31. Member 2 — HEIGHT ESTIMATION

Member 2 owns the algorithmic side of height estimation.

Potentially:

Object
+
Shadow length
+
Sonar geometry
↓
Approximate height

But:

insufficient geometry
↓
height = unavailable

This prevents fabricated measurements.

32. MEMBER 2 — HANDOFF TO MEMBER 3

Member 2 gives Member 3 modular functions such as:

preprocess()
shadow_analysis()
shape_analysis()
context_analysis()

```
geometry_analysis()  
evidence_fusion()  
calculate_reliability()  
estimate_size()  
estimate_height()
```

The outputs must use agreed JSON structures.

33. MEMBER 2 — WHAT THEY DO NOT OWN

Member 2 does **not** own:

- Model V1 training
- hard-negative training
- final model selection
- segmentation model training
- React dashboard
- FastAPI architecture
- Docker
- geographic UI/map rendering

This change is intentional so Member 2 can concentrate on the sonar-specific intelligence rather than trying to do half the AI and half the backend.

34. MEMBER 3 — SYSTEM INTEGRATION / BACKEND / GEOLOCATION

Role

System & Deployment Owner

Member 3 takes the outputs from the other members and makes them one usable application.

The original document already assigns Member 3 the complete integration, stability, error handling, performance and Docker responsibilities.

We extend that role to include the actual **application-side geolocation transformation and reporting**, removing this load from Member 2.

35. Member 3 — SYSTEM ARCHITECTURE

Member 3 defines:

```
frontend
  ↓
FastAPI
  ↓
processing pipeline
  ↓
AI model
  ↓
sonar evidence
  ↓
geolocation
  ↓
report
```

36. Member 3 — MODULE CONTRACTS

Member 3 creates the interfaces that everyone follows.

For example:

AI

```
detect(image)
```

Preprocessing

```
preprocess(image)
```

Evidence

```
analyze_candidate(image, detection)
```

Reliability

```
calculate_reliability(evidence)
```

Geolocation

```
geolocate(detection, metadata)
```

Reporting

```
generate_report(results)
```

This prevents everyone from writing incompatible code.

37. Member 3 — FASTAPI

Member 3 owns:

GET /health
POST /detect
POST /analyze
POST /segment
POST /report

Potential flow:

POST /analyze
↓
save upload
↓
preprocess
↓
AI detection
↓
sonar evidence
↓
reliability
↓
geolocation
↓
response

38. MEMBER 3 — GEOLOCATION

This is the revised allocation.

Member 2 produces sonar geometry information.

Member 3 turns available metadata into an application-ready geographic result.

Inputs:

bbox / sonar position
GPS
heading
range

timestamp
sensor offset
vehicle pose

Output:

```
{  
  "latitude": 10.52,  
  "longitude": 76.21,  
  "uncertainty_m": 5.3  
}
```

If the metadata does not support it:

```
{  
  "location_status": "unavailable"  
}
```

39. Member 3 — MAP/GIS INTEGRATION

Member 3 owns connecting the geographic output to the dashboard.

Example:

Detection
↓
lat/lon
↓
map marker

Map functionality should show:

- detection position
- object ID
- class
- reliability
- uncertainty if available

40. Member 3 — REPORTING PIPELINE

Member 2 provides structured evidence.

Member 3 turns all outputs into application reports.

Example:

```
{
  "object_id": "001",
  "class": "crab_pot",
  "detector_confidence": 0.87,
  "reliability_index": 0.82,
  "decision": "CONFIRM",
  "estimated_length_m": 2.4,
  "estimated_width_m": 1.1,
  "location": {
    "latitude": 10.52,
    "longitude": 76.21,
    "uncertainty_m": 5.3
  }
}
```

Then produce CSV.

41. MEMBER 3 — TRACKING / PERSISTENCE

Member 3 owns the software integration of tracking when sequential sonar data is available.

For example:

Frame 1 → candidate A
Frame 2 → candidate A
Frame 3 → candidate A

The system can associate them as one persistent target.

But if your dataset doesn't contain sequential observations, tracking is marked unavailable rather than fabricated.

42. MEMBER 3 — ERROR HANDLING

The final application must not crash when:

- image is invalid
- model file is missing
- metadata is absent
- GPS is unavailable
- unsupported input is provided
- no objects are detected
- multiple objects are detected

Instead:

No metadata

→ "Geolocation unavailable"

rather than:

application crash

43. MEMBER 3 — PERFORMANCE

Member 3 measures:

- end-to-end inference time

- API latency
- memory usage
- file processing time
- model loading time

They own system-level performance, while Member 1 owns model-level inference benchmarking.

44. MEMBER 3 — DOCKER

Member 3 packages:

Frontend

Backend

Model

Dependencies

The goal is:

`docker compose up`

starting the system consistently.

45. MEMBER 3 — WHAT THEY DO NOT OWN

Member 3 does not own:

- model training
- segmentation research
- shadow algorithm design
- reliability formula design
- UI visual design

They own **connecting and operating** those modules.

46. MEMBER 4 — UI + SYNTHETIC DATA

The original plan assigns Member 4 two main responsibilities: dashboard and Blender-based synthetic data generation.

We retain both.

47. MEMBER 4 — UI OWNERSHIP

Member 4 creates the user-facing dashboard.

The dashboard should ultimately provide:

Upload



Process



Results

and display:

Image

- original
- processed

AI

- bounding box
- class
- detector confidence

Evidence

- shadow
- shape
- context
- geometry

Decision

- Confirm
- Review
- Reject
- Unknown

Geospatial

- map
- coordinates
- uncertainty

Reports

- JSON
- CSV

48. MEMBER 4 — SYNTHETIC DATA

Member 4 can build synthetic scenes such as:

- pipes
- cylinders
- debris
- net-like objects
- structures

But:

Synthetic data is supporting training data, not proof of real-world performance.

Member 1 decides whether generated samples are suitable for training.

Member 4 does not decide that independently.

49. MEMBER 4 — UI API CONTRACT

Member 4 should not care how the AI internally works.

They need a clean response schema.

Example:

```
{
  "detections": [
    {
      "class": "crab_pot",
      "confidence": 0.84,
      "reliability": 0.81,
      "decision": "CONFIRM"
    }
  ]
}
```

Member 4 renders this.

50. MEMBER 4 — WHAT THEY DO NOT OWN

- AI training
- hard negatives
- preprocessing algorithms
- evidence fusion

- FastAPI architecture
- geolocation calculations
- Docker

51. MEMBER 5 — DATA + RESEARCH SUPPORT

The original plan assigns Member 5 dataset collection, organization, research and literature survey.

Their revised role stays simple.

52. MEMBER 5 — DATA RESPONSIBILITIES

Maintain:

Dataset name
Source
License
Number of images
Classes
Annotation type
Sensor/domain
Train/test split

They also help detect:

- duplicate images
- broken images
- missing labels
- suspicious labels
- inconsistent classes

53. MEMBER 5 — RESEARCH RESPONSIBILITIES

Research:

- SSS fundamentals
- acoustic shadows
- marine debris detection
- sonar object detection
- segmentation
- existing systems
- relevant datasets
- related academic work

The output should be a **research package**, not just random links.

54. MEMBER 5 — WHAT THEY DO NOT OWN

They don't decide:

- model architecture
- model training
- evidence weights
- geolocation algorithm
- API architecture

They provide information to the owners.

55. MEMBER 6 — DOCUMENTATION + QA

The original plan assigns Member 6 documentation, demo, coordination and submission support.

We keep this role focused.

56. MEMBER 6 — DOCUMENTATION

Maintain:

README

Technical architecture

Dataset documentation

Experiment log

Metrics

Known limitations

Setup instructions

Demo instructions

57. MEMBER 6 — QA

Member 6 should maintain a simple test checklist.

Examples:

Image uploads correctly

Model loads

Detection appears

No-detection case works

Multiple detections work

Missing GPS doesn't crash

JSON downloads

CSV downloads

Map works

This is not deep algorithmic testing.

It is **system acceptance testing**.

58. MEMBER 6 — DEMO

Create the final demonstration flow:

Upload SSS



Run analysis



Detection



Evidence



Reliability



Geolocation



Map



Report

59. MEMBER 6 — WHAT THEY DO NOT OWN

- AI implementation
- preprocessing
- backend implementation
- UI coding
- geolocation math

60. FINAL OWNERSHIP MATRIX

Work	M1	M2	M3	M4	M5	M6
Dataset ML preparation	✓ Owner				Support	
Class manifest	✓				Support	
Baseline YOLO	✓					
Error mining	✓				Support	
Hard negatives	✓				Support	
Fine-tuning V2	✓					
Segmentation	✓					
Augmentation	✓					
AI evaluation	✓					
ONNX	✓					
TensorRT benchmark	✓					
Preprocessing		✓				
Denoising		✓				
CLAHE		✓				
Shadow analysis		✓				
Shape analysis		✓				
Geometry analysis		✓				
Context analysis		✓				
Evidence fusion		✓				
Reliability Index		✓				
Confirm/Review/Reject		✓				

Unknown/review pathway	✓					
Size estimation	✓					
Height estimation	✓					
API architecture		✓				
Geolocation transform		✓				
GIS integration		✓				
Tracking integration		✓				
JSON/CSV pipeline		✓				
FastAPI		✓				
Docker		✓				
UI			✓			
Synthetic data			✓			
Dataset research				✓		
Literature research				✓		
Documentation					✓	
QA checklist					✓	
Demo					✓	

61. THE THREE CORE OWNERS NOW

This is the most important team understanding.

MEMBER 1

"I own the AI."

Dataset
↓
Model V1
↓
Error analysis
↓
Hard negatives
↓
Model V2
↓
Segmentation
↓
Evaluation
↓
Final model

MEMBER 2

"I own the sonar intelligence."

Image
↓
Preprocessing
↓
Shadow
↓
Shape
↓
Context
↓
Geometry
↓
Evidence Fusion
↓
Reliability

MEMBER 3

"I own the product."

AI
+
Sonar Intelligence
+
Geolocation
+
API
+
UI
+
Reports
+
Docker
↓
WORKING APPLICATION

62. THE CRITICAL HANDOFFS

M1 → M2

Member 1 does **not hand over the entire model development responsibility**.

They hand over:

model predictions
error cases
model version information

because M1 continues owning AI training.

M1 → M3

Final:

final_model.pt
final_model.onnx
classes.yaml

M2 → M3

Final:

preprocessing
evidence
reliability
size
height

M4 → M3

frontend

M5 → M1/M6

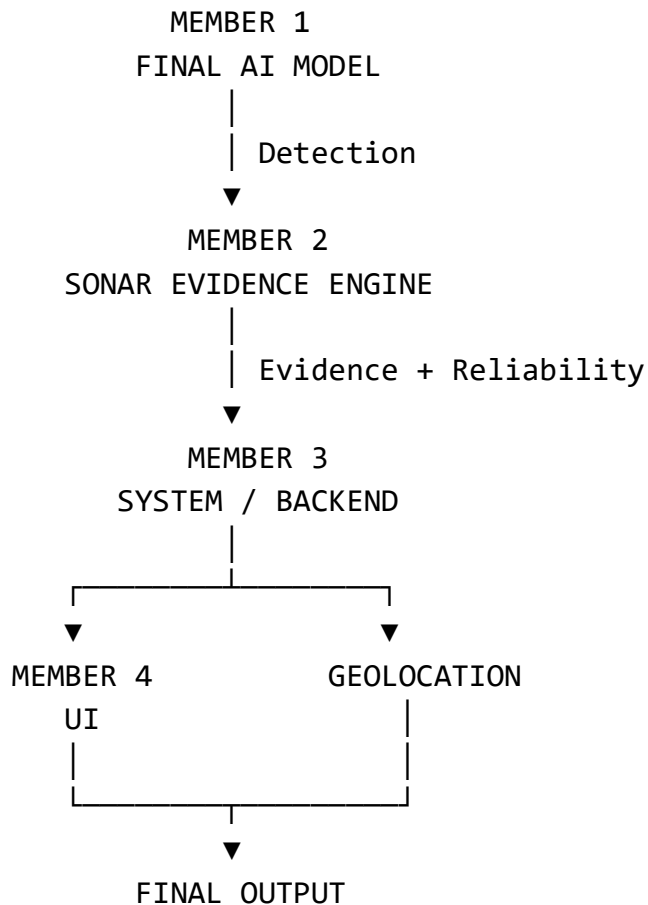
dataset information
research material

M6 → whole team

test records
documentation
demo materials

63. HOW THE WORK CONNECTS WITHOUT DUPLICATION

The system should function like:



Nobody trains the same model twice independently.

Nobody writes two different shadow systems.

Nobody calculates GPS in three different places.

Each capability has one owner.

64. "DONE" CRITERIA FOR EACH MEMBER

MEMBER 1 is DONE when

There is a reproducible final model:

image

↓

prediction

with documented metrics and a versioned checkpoint.

MEMBER 2 is DONE when

Given a detection:

image + bbox

↓

evidence

↓

Reliability Index

works reproducibly.

MEMBER 3 is DONE when

upload

↓

complete pipeline

↓

result

works from one application.

MEMBER 4 is DONE when

The UI correctly displays the actual backend outputs.

MEMBER 5 is DONE when

The team knows exactly which datasets, labels and sources it is using.

MEMBER 6 is DONE when

The documentation and test evidence match what the system actually does.

65. WHAT SHOULD NOT BE SHARED AS "EVERYONE'S RESPONSIBILITY"

Avoid phrases like:

"Everyone will work on the AI."

Instead:

Member 1 owns AI.

Avoid:

"Everyone will help with preprocessing."

Instead:

Member 2 owns preprocessing. Others can report bugs.

Avoid:

"Everyone will do integration."

Instead:

Member 3 owns integration.

This is how you prevent six people from modifying the same files or duplicating work.

66. THE FINAL ROLE DESCRIPTIONS YOU CAN SEND INDIVIDUALLY

MEMBER 1

AI/ML Lead: You own the complete AI lifecycle. Prepare the ML datasets, build and train the baseline detector, produce Model V1, analyze errors, create hard negatives and difficult positives, fine-tune Model V2, handle segmentation, run all model evaluations, perform ablation studies and export the final model to ONNX/TensorRT where practical. Your output to the rest of the team is a clean, versioned model and documented metrics.

MEMBER 2

Sonar Intelligence Lead: You own the sonar-specific intelligence layer. Build preprocessing, denoising, normalization, CLAHE, acoustic-shadow analysis, shape/geometry analysis, context analysis, evidence fusion and the Reliability Index. Handle Confirm/Review/Reject and the unknown/review pathway. Estimate size and height only when sonar geometry supports them. You do not handle hard-negative training.

MEMBER 3

System Integration Lead: You own the actual software product. Build FastAPI, define module contracts, connect the final AI model and sonar-intelligence modules, handle tracking where sequential data exists, implement geolocation transformation and uncertainty output using available metadata, generate report responses, connect the dashboard, handle error states, optimize performance and package the system with Docker.

MEMBER 4

UI + Synthetic Data: Build the dashboard and the optional synthetic sonar-like dataset. Your UI must display actual system results, not invented values. Your synthetic data is supplied to Member 1 only after its format and quality are checked.

MEMBER 5

Data + Research: Own dataset sourcing, metadata, licenses, class verification and literature/research support. Maintain the dataset inventory and help the technical team verify difficult samples.

MEMBER 6

Documentation + QA: Own README, technical documentation, experiment records, testing checklist, screenshots, demo and submission materials. Your job is to make sure the documented system exactly matches the system that was actually built.

STAGE 2 CHECKPOINT

This is now the **final ownership structure** I recommend:

MEMBER 1

AI + TRAINING + HARD NEGATIVES + SEGMENTATION



MEMBER 2

SONAR PROCESSING + EVIDENCE + RELIABILITY



MEMBER 3

BACKEND + GEOLOCATION + INTEGRATION + DEPLOYMENT



MEMBER 4

UI + SYNTHETIC DATA

MEMBER 5

DATA + RESEARCH

MEMBER 6

DOCUMENTATION + QA

This keeps the original source structure of Members 1–3 as the heavy technical core while removing the biggest overload from Member 2 and making the AI lifecycle a single coherent ownership chain under Member 1.

STAGE 3 — COMPLETE SONAR-SHIELD FEATURE SPECIFICATION

This stage defines **what the final product should contain**.

We are not doing the day-by-day schedule yet.

We are not splitting the work further than the ownership already approved in Stage 2.

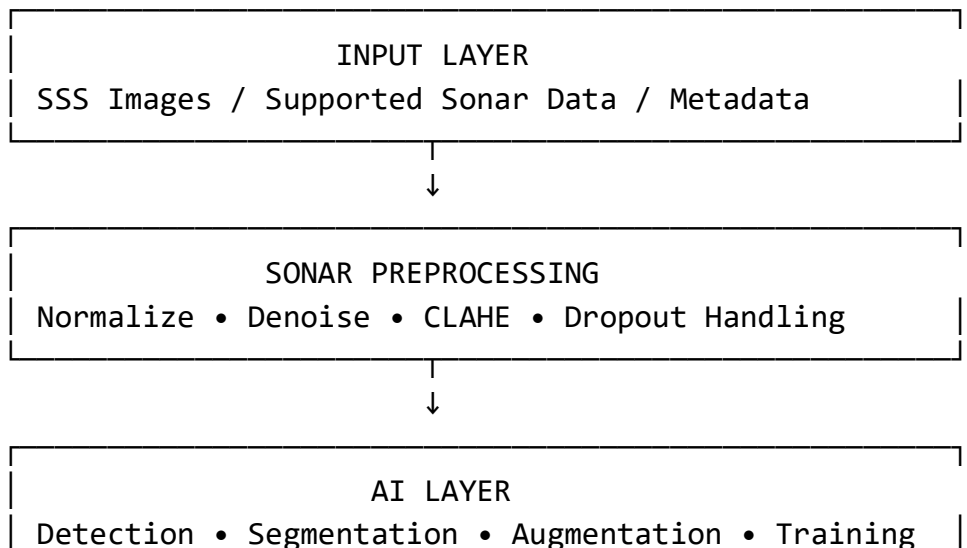
For **every feature**, I'll define:

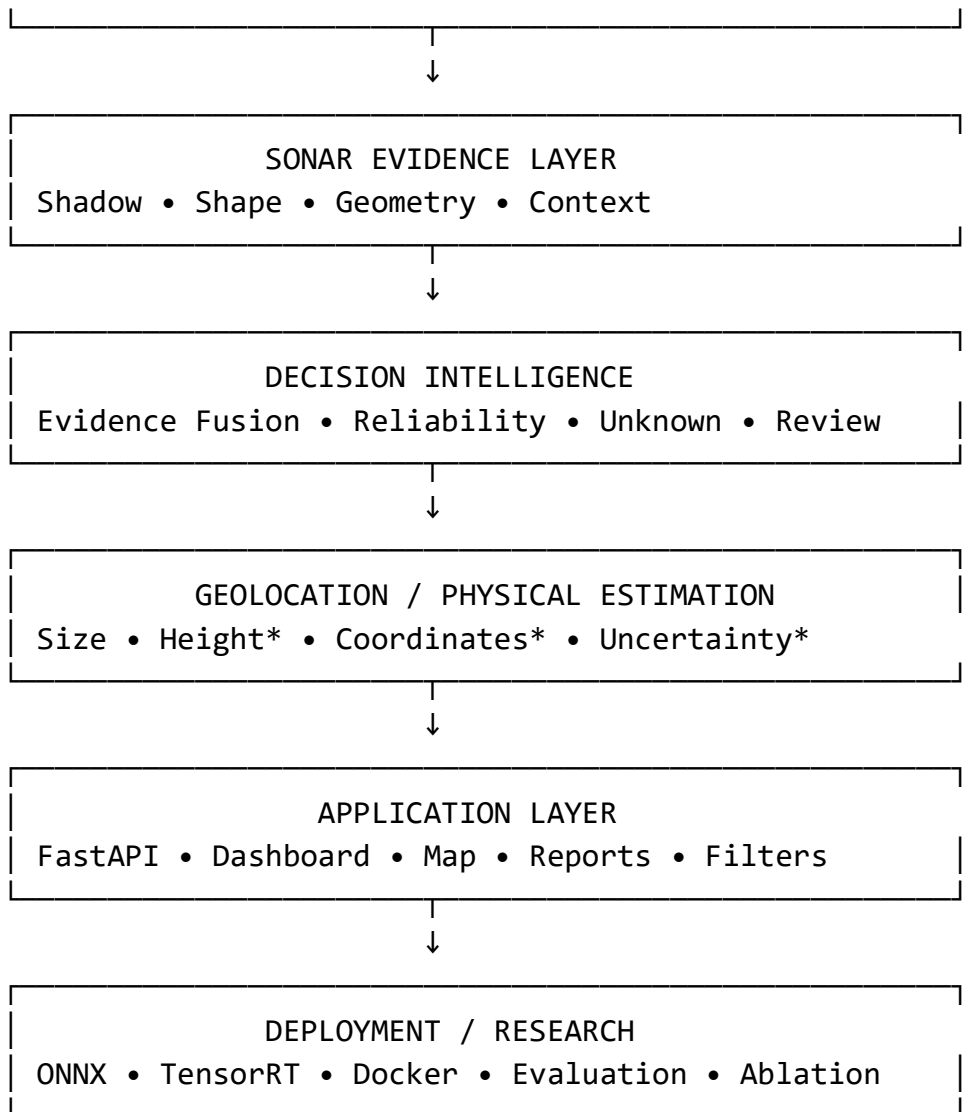
What it does → Why we need it → Input → Output → Implementation idea → Owner → Dependency → Important limitation

The feature set below incorporates the original project plan's detection/segmentation, preprocessing, confidence/filtering, geotagging, API, dashboard, synthetic-data and deployment requirements.

1. COMPLETE PRODUCT FEATURE MAP

SONAR-SHIELD should ultimately have these layers:





* means the feature is **conditional on sufficient sonar/navigation metadata and geometry**.

PART A — INPUT FEATURES

Feature 1 — SSS Image Input

What it does

Allows the system to receive a Side-Scan Sonar image.

Typical input:

PNG

JPG/JPEG

TIFF

depending on the dataset/source.

Why

This is the basic input for the AI pipeline.

Input

sonar image

Output

validated image object

Owner

Member 3 — input/API handling.

Member 1 consumes the image for AI.

Dependency

None.

Important rule

We should never assume every sonar image has identical:

- resolution
- intensity range
- dimensions
- orientation

The preprocessing layer must handle variation.

Feature 2 — Sonar Metadata Input

What it does

Allows the system to receive metadata associated with the sonar acquisition.

Potential fields:

GPS
heading
range
timestamp
ping information
sensor position
vehicle pose
sensor offset

Why

Metadata is needed for physical/geospatial interpretation.

Owner

Member 3

Output

A normalized metadata object:

```
{  
  "gps": {},  
  "heading": null,  
  "range_m": null,  
  "timestamp": null  
}
```

Limitation

Not every dataset will contain all fields.

The system must support:

```
metadata complete  
metadata partial  
metadata unavailable
```

Feature 3 — Input Validation

What it does

Checks whether the uploaded file is usable.

Checks:

- valid file
- supported image format
- readable image
- dimensions
- grayscale/intensity information
- metadata presence
- corrupted file

Owner

Member 3

Output

Either:

VALID INPUT

or:

INVALID / UNSUPPORTED INPUT

Why

A judge uploading a bad file shouldn't crash the application.

PART B — PREPROCESSING FEATURES

Feature 4 — Intensity Normalization

What it does

Normalizes sonar intensity values into a more consistent range.

Conceptually:

Raw intensity

↓

Normalization

↓

consistent numerical representation

Why

Different sonar captures may have different intensity distributions.

Owner

Member 2

Input

Raw SSS image.

Output

Normalized SSS image.

Feature 5 — Speckle / Noise Reduction

What it does

Reduces unwanted sonar noise while attempting to preserve target structure.

Potential methods:

- Lee
- Frost
- Non-Local Means
- other filtering methods

The original team plan specifically identifies Lee/Frost/Non-Local Means-type noise reduction.

Owner

Member 2

Important

Do not automatically assume stronger denoising is better.

The output needs to be tested against detection performance.

Feature 6 — CLAHE / Local Contrast Enhancement

What it does

Improves local contrast in the sonar image.

Owner

Member 2

Input

Normalized/noisy image.

Output

Locally enhanced image.

Important

We must experimentally compare:

raw → detector

versus:

processed → detector

rather than claiming CLAHE automatically improves detection.

Feature 7 — Data Dropout Handling

What it does

Detects regions where sonar data is missing, corrupted or unreliable.

Owner

Member 2

Possible output

```
{  
  "dropout_detected": true,  
  "affected_region": [...]  
}
```

Why

Missing acoustic data should not be interpreted as an ordinary seabed region.

Feature 8 — Shadow-Preserving Preprocessing

What it does

Ensures preprocessing does not unintentionally erase acoustic-shadow information.

Owner

Member 2

Important

The goal is not:

"Remove dark regions."

The goal is:

Preserve meaningful acoustic structure while reducing irrelevant noise.

PART C — AI MODEL FEATURES

Feature 9 — Object Detection

Core AI feature

This is the first major AI capability.

The model outputs:

```
class
bounding box
detector confidence
```

Example:

```
{
  "class": "crab_pot",
  "confidence": 0.87,
  "bbox": [120, 200, 300, 360]
}
```

Owner

Member 1

Why

This creates candidate objects for the rest of SONAR-SHIELD.

The existing plan identifies object detection as a central AI/ML responsibility.

Feature 10 — Multi-Class Detection

The detector can support multiple classes **only where corresponding labelled data exists**.

Potential example:

crab_pot
pipeline
shipwreck
other supported anomaly

Owner

Member 1

Important rule

We do **not** create classes merely because they sound useful.

Every class must have:

actual labels
+
appropriate data

Feature 11 — Segmentation

What it does

Produces a pixel-level mask for supported objects.

```
image
↓
segmentation
↓
object mask
```

Owner

Member 1

Why

Useful for:

- exact object boundaries
- geometry
- shape analysis
- certain datasets where masks exist

Important

Segmentation requires pixel-level labelled masks.

Detection boxes alone are not enough.

Feature 12 — Model Confidence

What it does

Stores the detector's own confidence.

Example:

Crab Pot

Detector Confidence = 0.86

Owner

Member 1

Important

This is **not the final SONAR-SHIELD Reliability Index**.

Feature 13 — Model Augmentation

What it does

Improves generalization through controlled training variation.

Potential augmentations:

- intensity variation
- noise
- mild blur
- geometric variation
- sonar-like disturbances
- shadow-related variations

Owner

Member 1

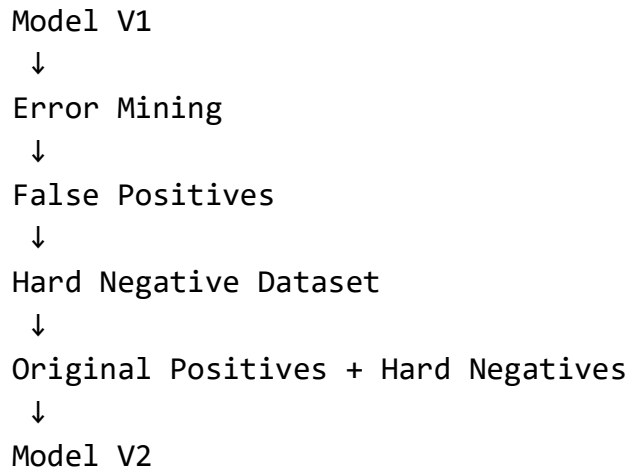
The original plan explicitly includes speckle-noise simulation, acoustic shadows, geometric distortions and intensity variation.

Feature 14 — Hard-Negative Learning

What it does

Improves the detector using false-positive examples.

Flow:



Owner

Member 1

Important

Hard negatives are generally background samples, not new classes.

Feature 15 — Difficult-Positive Learning

What it does

Preserves examples where the model missed real targets.

Examples:

small target
weak target
partially obscured target
low-contrast target

Owner

Member 1

Why

Hard-negative learning reduces false positives, but difficult positives help address false negatives.

Feature 16 — Model Versioning

What it does

Keeps:

V1
V2
V3
Final

separately.

Owner

Member 1

Why

We need to know exactly which model produced which result.

Example:

model_v1.pt
model_v2.pt
final_model.pt

Feature 17 — Experiment Logging

For every significant training experiment:

experiment ID
dataset
model
parameters
augmentation
metrics
observations

Owner

Member 1

Why

Prevents:

"Which model did we train yesterday?"

PART D — SONAR EVIDENCE FEATURES

Feature 18 — Candidate ROI Expansion

What it does

After YOLO detects:

bounding box

the system expands the surrounding region.

Why?

Because the acoustic shadow may extend **outside the object bounding box**.

Owner

Member 2

Flow

YOLO bbox

↓

expanded ROI

↓

sonar evidence analysis

Feature 19 — Acoustic Shadow Detection

What it does

Searches for a candidate acoustic shadow associated with the detected object.

Potential processing:

ROI

↓

intensity analysis

↓

thresholding / morphology

↓

connected regions

↓

shadow candidate

Output

```
{  
  "shadow_detected": true,  
  "shadow_length_px": 82,  
  "shadow_score": 0.74  
}
```

Owner

Member 2

Feature 20 — Shadow Direction Analysis

What it does

Determines the orientation of the candidate shadow.

Why

A shadow should be interpreted in the context of sonar acquisition geometry.

Owner

Member 2

Important

Direction is evidence, not proof.

Feature 21 — Shadow/Object Relationship

What it does

Compares:

object dimensions
shadow dimensions
shadow direction

Possible feature:

shadow-to-object ratio

Owner

Member 2

Why

The object and shadow should be interpreted together rather than independently.

Feature 22 — Object Shape Analysis

What it does

Extracts:

- width
- height
- area
- aspect ratio
- perimeter
- compactness
- orientation
- elongation

Owner

Member 2

Why

Provides additional evidence to the decision engine.

Feature 23 — Seabed Context Analysis

What it does

Analyzes the neighborhood around the candidate.

Potential features:

- local mean intensity
- variance
- texture
- contrast
- surrounding seabed statistics

Owner

Member 2

Question

Does the candidate differ significantly from its immediate environment?

Feature 24 — Geometry Evidence

What it does

Combines geometric relationships between:

object
shadow
surrounding region

Owner

Member 2

Why

It strengthens the sonar-specific evidence layer.

Feature 25 — Evidence Fusion

This is one of the **core SONAR-SHIELD features**.

Inputs

AI confidence
shadow score
shape evidence
context evidence
geometry

Output

Evidence vector

Example:

```
{  
  "ai": 0.84,  
  "shadow": 0.76,  
  "shape": 0.81,  
  "context": 0.68  
}
```

Owner

Member 2

PART E — DECISION FEATURES

Feature 26 — SONAR-SHIELD Reliability Index

What it does

Combines the evidence into one system-level score.

Conceptually:

AI
+
Shadow
+
Shape
+
Context
+
Geometry
↓
Reliability Index

Owner

Member 2

Important terminology

Call it:

Reliability Index

not automatically:

"probability."

Unless properly calibrated, it's an engineering decision score.

Feature 27 — Confirm / Review / Reject

What it does

Converts the Reliability Index into a decision state.

high reliability

↓

CONFIRM

intermediate

↓

REVIEW

low

↓

REJECT

Owner

Member 2

Important

Thresholds should be tuned/validated rather than chosen arbitrarily.

Feature 28 — Unknown / Low-Confidence Anomaly Path

What it does

Handles candidates that do not confidently belong to a known trained class.

Output:

UNKNOWN / REVIEW REQUIRED

Owner

Member 2

Important

This is a **review pathway**, not a claim of full open-set recognition.

Feature 29 — Human Review Information

For review candidates, the dashboard should show:

object image
detector confidence
shadow evidence
shape evidence
context

reliability
reason/status

Owner

Member 2 defines the information.

Member 4 displays it.

Member 3 connects it.

Feature 30 — Human Feedback Hook

What it does

Allows a reviewer eventually to mark:

confirmed
false positive
unknown
wrong class

Owner

Member 3 for application/API.

Why

Those labels can later become additional training information.

This supports the longer-term learning loop:

Detection

↓

Human Review

↓

Corrected Label



Future Training

This is a future-facing feature unless time permits a basic implementation.

PART F — TRACKING FEATURES

Feature 31 — Detection Persistence

What it does

Checks whether an object appears consistently over multiple sequential sonar frames/pings.

Example:

Frame 1 → object

Frame 2 → object

Frame 3 → object

Owner

Member 3

Important

Only enable this when sequential/overlapping sonar data exists.

For disconnected still images:

tracking = unavailable

Feature 32 — Object Association / Tracking

What it does

Associates the same detected target between sequential frames.

Potential methods:

- IoU matching
- centroid matching
- ByteTrack-type tracker

Owner

Member 3

Output

Object ID: 001

appeared in frames 31–37

PART G — PHYSICAL ESTIMATION FEATURES

Feature 33 — Object Size Estimation

What it does

Converts image/sonar dimensions into approximate physical dimensions.

bbox pixels

↓

sonar scale/range

↓

metres

Owner

Member 2

Output

Estimated length

Estimated width

Important

Only valid when calibrated sonar scale/range information exists.

Feature 34 — Object Height Estimation

What it does

Uses shadow geometry when suitable metadata/geometry is available.

Conceptually:

shadow length

+

sonar geometry

↓

estimated object height

Owner

Member 2

Important

Conditional feature.

Output may legitimately be:

Height unavailable

Feature 35 — Physical Measurement Uncertainty

What it does

Adds uncertainty to size/height estimates.

Why

Measurements can depend on:

- sonar resolution
- calibration
- shadow boundary
- geometry
- object orientation

Owner

Member 2

PART H — GEOLOCATION FEATURES

Feature 36 — Sonar Coordinate Extraction

What it does

Converts image-space/object position into sonar-relative coordinates.

Owner

Member 3, using geometry information supplied by Member 2.

Feature 37 — Navigation Metadata Parsing

What it does

Reads:

GPS
heading
range
timestamp
sensor offset
vehicle pose

Owner

Member 3

Feature 38 — Geographic Transformation

What it does

Transforms the detection into:

latitude
longitude

when the metadata supports it.

Owner

Member 3

Output

```
{  
  "latitude": 10.52,  
  "longitude": 76.21  
}
```

Feature 39 — Location Uncertainty

What it does

Adds uncertainty to location.

Example:

Latitude: X

Longitude: Y

Uncertainty: ± 5.3 m

Owner

Member 3

Why

Prevents the dashboard from presenting approximate sonar localization as exact.

Feature 40 — Missing Metadata Handling

What it does

If GPS or necessary geometry is absent:

Geolocation unavailable

instead of generating fake coordinates.

Owner

Member 3

This is an important robustness feature.

PART I — REPORTING FEATURES

Feature 41 — JSON Report Generation

What it does

Produces machine-readable structured output.

Example:

```
{  
  "object_id": "001",  
  "class": "crab_pot",  
  "detector_confidence": 0.87,  
  "reliability_index": 0.82,  
  "decision": "CONFIRM",  
  "estimated_length_m": 2.4,  
  "estimated_width_m": 1.1,  
  "latitude": 10.52,
```

```
"longitude": 76.21,  
"uncertainty_m": 5.3  
}
```

Owner

Member 3

Feature 42 — CSV Report Generation

What it does

Creates a tabular report suitable for spreadsheet/GIS workflows.

Example:

ID	Class	Detector	Reliability	Decision	Length	Latitude	Longitude
001	Crab Pot	.87	.82	Confirm	2.4	10.52	76.21

Owner

Member 3

Feature 43 — Report Metadata

Reports should include:

- model version
- processing version
- timestamp
- input filename
- location status
- uncertainty status
- evidence values

Owner

Member 3

PART J — DASHBOARD FEATURES

Feature 44 — Image Upload

Owner

Member 4

API integration

Member 3

Feature 45 — Original vs Processed Viewer

Show:

Original

Processed

Owner

Member 4

Feature 46 — Detection Overlay

Show:

bounding box
class
detector confidence

Owner

Member 4

Feature 47 — Segmentation Overlay

Where segmentation exists:

pixel mask

Owner

Member 4

Feature 48 — Evidence Panel

Display:

Detector Confidence
Shadow Evidence
Shape Evidence
Context Evidence
Reliability Index

Owner

Member 4

Feature 49 — Decision Panel

Display:

CONFIRM

REVIEW

REJECT

UNKNOWN

with appropriate explanation/status.

Owner

Member 4

Feature 50 — Class Filter

The UI can filter displayed detections by class.

This is part of the original dashboard concept.

Owner

Member 4

Feature 51 — Reliability Filter

Allow users to filter results based on reliability thresholds.

Owner

Member 4

Backend support

Member 3

Feature 52 — Interactive Map

Display detections with:

marker
class
reliability
object ID
uncertainty

Owner

Member 4 for display.

Member 3 for geographic data/API.

Feature 53 — Detection Detail View

Selecting a marker/object should show:

Object ID
Class
Confidence
Reliability
Evidence
Dimensions
Location

Owner

Member 4

Feature 54 — Download JSON/CSV

Buttons:

Download JSON

Download CSV

Owner

Member 4 UI.

Member 3 generation/API.

PART K — SYNTHETIC DATA FEATURES

Feature 55 — Blender Scene Generation

Owner

Member 4

Generate scenes containing:

- pipes
- cylinders
- debris
- structures
- net-like forms where practical

This comes directly from the original team plan's synthetic-data responsibility.

Feature 56 — Synthetic Sonar-Like Rendering

Owner

Member 4

The objective is to create **sonar-like training examples**, not claim physically perfect acoustic simulation.

Possible components:

- grayscale appearance
- shadows
- noise
- varying intensity
- object placement

Feature 57 — Synthetic Annotation Generation

Each synthetic image should have:

image
+
bounding box / available label

Owner

Member 4

Feature 58 — Synthetic Data Validation

Before Member 1 trains on it:

- inspect images
- verify labels
- verify object visibility
- remove unusable examples

Owner

Member 1 decides whether it is suitable for training.

Member 4 performs the creation/cleanup.

PART L — EVALUATION FEATURES

Feature 59 — Detection Metrics

Owner

Member 1

Required:

- Precision
- Recall
- F1
- [mAP@0.5](#)
- [mAP@0.5:0.95](#)
- inference speed

These are explicitly included in the original team plan.

Feature 60 — Segmentation Metrics

Owner

Member 1

Potential:

- IoU
- Dice
- mask visual inspection

Feature 61 — False-Positive Benchmark

Create a negative-only evaluation set.

Measure:

false detections

Owner

Member 1

Feature 62 — Cross-Domain Evaluation

Test where possible across:

different site
different sonar

different survey
different resolution

Owner

Member 1

Why

This measures domain robustness.

Feature 63 — Resolution Robustness

Test images with different spatial dimensions/resolutions.

Owner

Member 1 for AI behavior.

Member 2 for preprocessing behavior.

Feature 64 — Missing-Metadata Robustness

Run:

metadata complete
metadata partial
metadata missing

Owner

Member 3

Expected behavior

Detection may still work.

Location may become:

unavailable

rather than crashing.

Feature 65 — Ablation Study

Compare:

Baseline

+ preprocessing

+ hard negatives

+ evidence fusion

+ full system

Owner

Member 1 for experiment design/model metrics.

Member 2 contributes the evidence-fusion component.

This is important for demonstrating that SONAR-SHIELD's additional components actually matter.

Feature 66 — Failure Analysis

Create categories:

False Positive

False Negative

Class Confusion

Weak Shadow

Bad Metadata

Noise

Unknown

Owner

Member 1 for AI errors.

Member 2 for sonar/evidence errors.

PART M — DEPLOYMENT FEATURES

Feature 67 — ONNX Export

Owner

Member 1

PyTorch model

↓

ONNX

Feature 68 — ONNX Inference Verification

Compare:

PyTorch output

vs

ONNX output

Owner

Member 1

Feature 69 — TensorRT Benchmark

Owner

Member 1

Where suitable NVIDIA hardware is available.

Measure:

- latency
- FPS
- memory

Do not claim improvements without measured evidence.

Feature 70 — Docker Packaging

Owner

Member 3

Package:

frontend

backend

model

dependencies

Feature 71 — Health/Status API

Owner

Member 3

Example:

GET /health

Returns:

```
{
  "status": "ok",
  "model_loaded": true
}
```

Feature 72 — Error Handling

Owner

Member 3

Handle:

- invalid image
- missing model
- no detection
- missing metadata
- unsupported input
- API error
- inference failure

Feature 73 — Logging

Owner

Member 3

Record:

request
model version
processing result
errors
timing

Useful for debugging and demonstrations.

PART N — RESEARCH / MAINTENANCE FEATURES

Feature 74 — Model Version Registry

Track:

V1
V2
Final

with:

dataset
metrics
date
training configuration

Owner

Member 1

Feature 75 — Dataset Registry

Track:

dataset name

source

license

class

annotation

domain

version

Owner

Member 5

Feature 76 — Experiment Registry

Store:

Experiment ID

Model

Dataset

Preprocessing

Augmentation

Metrics

Conclusion

Owner

Member 1

Feature 77 — Evidence Audit Trail

For each detection, retain:

AI score
shadow score
shape score
context score
reliability
decision

Owner

Member 2

Integration

Member 3

This makes the system more explainable.

78. COMPLETE FEATURE-TO-OWNER MATRIX

#	Feature	Owner
1	SSS image input	M3
2	Metadata input	M3
3	Input validation	M3
4	Intensity normalization	M2

5	Noise reduction	M2
6	CLAHE	M2
7	Dropout handling	M2
8	Shadow-preserving preprocessing	M2
9	Object detection	M1
10	Multi-class detection	M1
11	Segmentation	M1
12	Detector confidence	M1
13	Data augmentation	M1
14	Hard-negative learning	M1
15	Difficult-positive learning	M1
16	Model versioning	M1
17	Experiment logging	M1
18	ROI expansion	M2
19	Shadow detection	M2
20	Shadow direction	M2
21	Object-shadow relationship	M2
22	Shape analysis	M2
23	Context analysis	M2
24	Geometry evidence	M2
25	Evidence fusion	M2
26	Reliability Index	M2
27	Confirm/Review/Reject	M2
28	Unknown/review path	M2
29	Human-review info	M2/M4
30	Human feedback hook	M3
31	Persistence	M3
32	Tracking	M3
33	Size estimation	M2
34	Height estimation	M2
35	Measurement uncertainty	M2
36	Sonar coordinate extraction	M3
37	Metadata parsing	M3
38	Geographic transformation	M3
39	Location uncertainty	M3
40	Missing metadata handling	M3
41	JSON reports	M3

42	CSV reports	M3
43	Report metadata	M3
44	Upload UI	M4
45	Original/processed viewer	M4
46	Detection overlay	M4
47	Segmentation overlay	M4
48	Evidence panel	M4
49	Decision panel	M4
50	Class filter	M4
51	Reliability filter	M4
52	Interactive map	M4/M3
53	Detection detail view	M4
54	Report downloads	M4/M3
55	Blender scenes	M4
56	Synthetic sonar-like rendering	M4
57	Synthetic labels	M4
58	Synthetic-data validation	M1/M4
59	Detection metrics	M1
60	Segmentation metrics	M1
61	FP benchmark	M1
62	Cross-domain evaluation	M1
63	Resolution robustness	M1/M2
64	Missing-metadata robustness	M3
65	Ablation	M1/M2
66	Failure analysis	M1/M2
67	ONNX export	M1
68	ONNX verification	M1
69	TensorRT benchmark	M1
70	Docker	M3
71	Health API	M3
72	Error handling	M3
73	Logging	M3
74	Model registry	M1
75	Dataset registry	M5
76	Experiment registry	M1
77	Evidence audit trail	M2/M3

79. FEATURES THAT ARE CORE VS CONDITIONAL

This distinction is important because otherwise the team may think every feature is equally mandatory.

CORE PRODUCT FEATURES

These define SONAR-SHIELD:

- Detection
- Preprocessing
- Shadow analysis
- Shape/geometry
- Context
- Evidence fusion
- Reliability Index
- Confirm/Review/Reject
- Unknown/review path
- JSON/CSV
- Dashboard

IMPORTANT AI FEATURES

- Baseline model
- Hard-negative learning
- Difficult-positive learning
- Segmentation
- Augmentation
- Evaluation
- Ablation
- Failure analysis

CONDITIONAL PHYSICAL/GEOSPATIAL FEATURES

Size
Height
Geolocation
Location uncertainty
Tracking

They depend on the data/metadata available.

DEPLOYMENT FEATURES

ONNX
TensorRT
Docker

These make the system deployable, but they shouldn't dictate the scientific design.

80. FEATURES WE SHOULD NOT CLAIM WITHOUT VALIDATION

This is critical.

Don't say:

"Exact object height."

Say:

"Estimated height when sufficient sonar geometry is available."

Don't say:

"Exact location."

Say:

"Estimated geographic location with uncertainty when navigation metadata is available."

Don't say:

"Full unknown-object detection."

Say:

"Unknown/low-confidence anomaly review pathway."

Don't say:

"Physically simulated sonar."

Say:

"Synthetic sonar-like training data."

Don't say:

"Reliability is 82% probability."

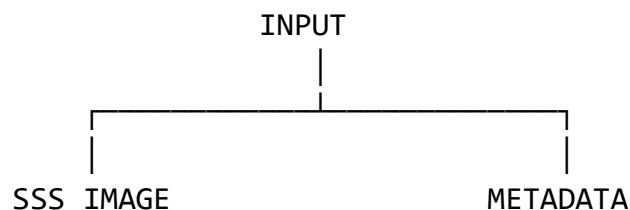
Say:

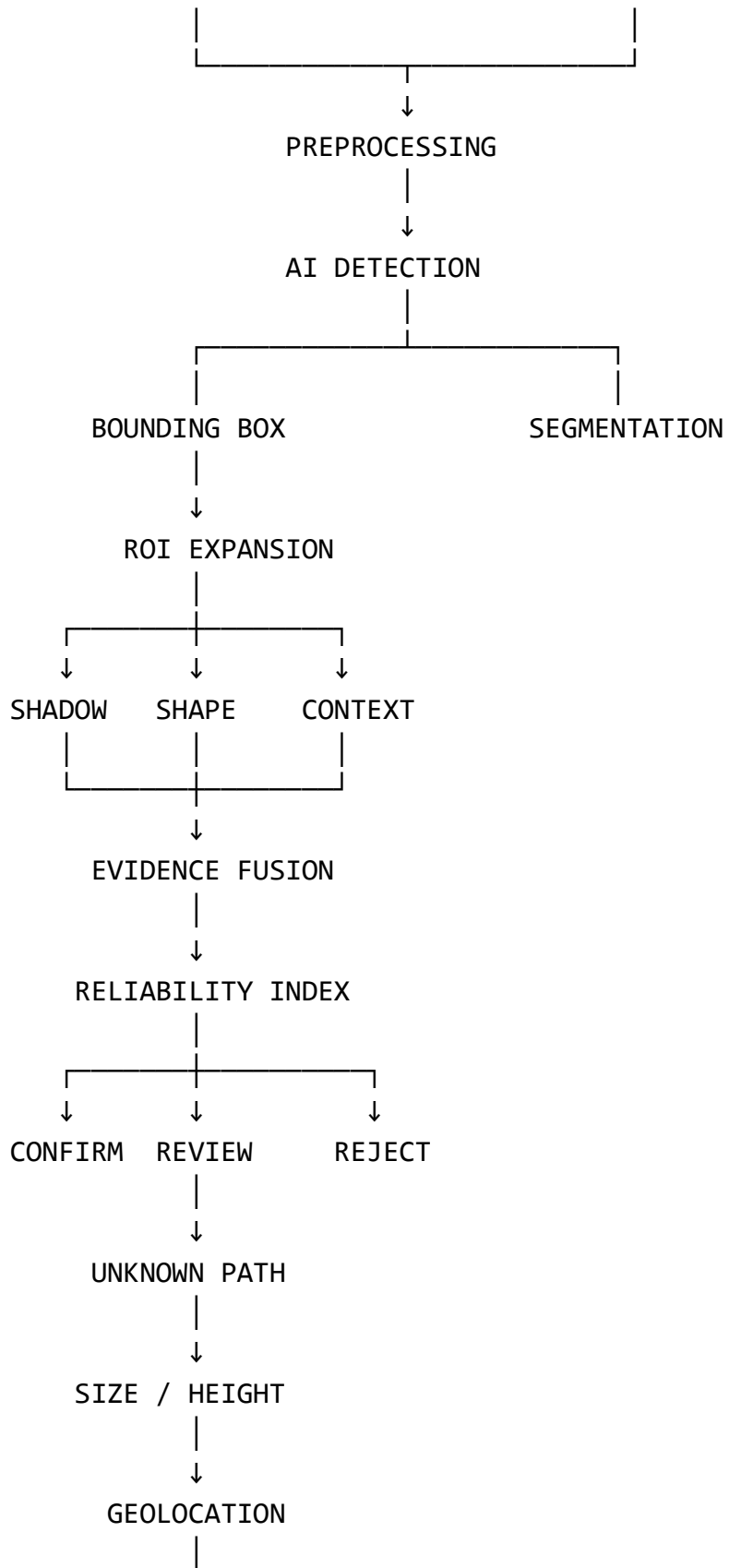
"Reliability Index = 0.82."

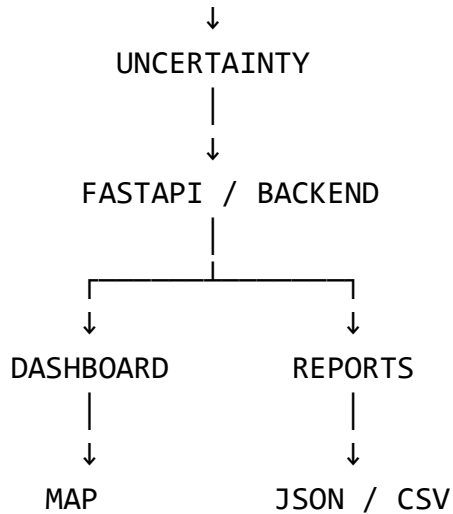
These distinctions make the product technically much safer.

81. THE COMPLETE FEATURE FLOW

When everything is connected:







That is the **feature architecture** we should build from.

82. THE MOST IMPORTANT PRODUCT INSIGHT

The project is not:

YOLO + UI + map

It is:

YOLO

+

SONAR-SPECIFIC EVIDENCE

+

RELIABILITY DECISION

+

UNCERTAINTY-AWARE OUTPUT

That is where the SONAR-SHIELD concept lives.

The original team plan already frames Member 1 around the detection/segmentation model, Member 2 around preprocessing/confidence/geotagging, Member 3 around integration, and Member 4 around the dashboard/synthetic data. The revised feature

ownership above makes those boundaries more explicit and keeps the hard-negative learning loop entirely with Member 1.

STAGE 3 CHECKPOINT

The feature set is now defined as:

Input → preprocessing → AI → sonar evidence → reliability → physical/geospatial estimation → API → dashboard → reports → deployment/evaluation.

STAGE 4 — COMPLETE MILESTONE PLAN

This stage converts the **77-feature specification from Stage 3** into a set of **concrete development milestones**.

The purpose of a milestone is not simply:

"Work on preprocessing."

A proper milestone means:

A specific technical component is implemented, tested, documented, and produces a defined output that the next component can consume.

The original project plan describes the overall sequence as data → model → preprocessing/confidence/geotagging → UI → integration → testing/submission.

For our revised structure, we keep that architecture but make the handoffs and completion criteria much stricter.

1. THE COMPLETE MILESTONE STRUCTURE

We will use **16 major milestones**:

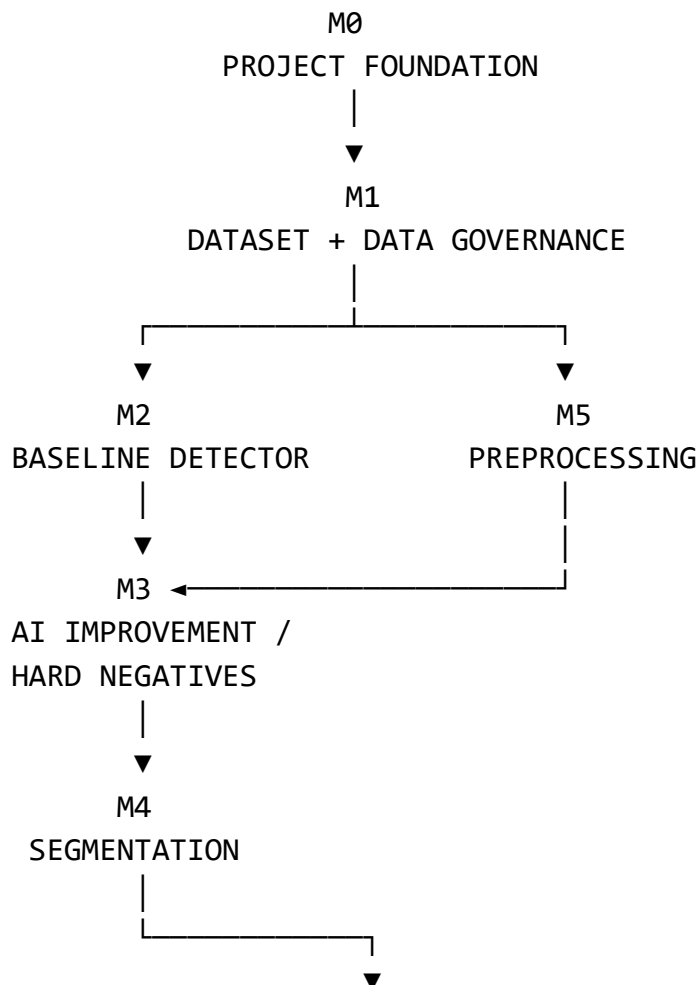
- M0 – Project Foundation
- M1 – Dataset & Data Governance
- M2 – Baseline Detection Model
- M3 – AI Improvement & Hard-Negative Learning
- M4 – Segmentation Branch
- M5 – Sonar Preprocessing
- M6 – Shadow Analysis
- M7 – Shape / Geometry / Context
- M8 – Evidence Fusion

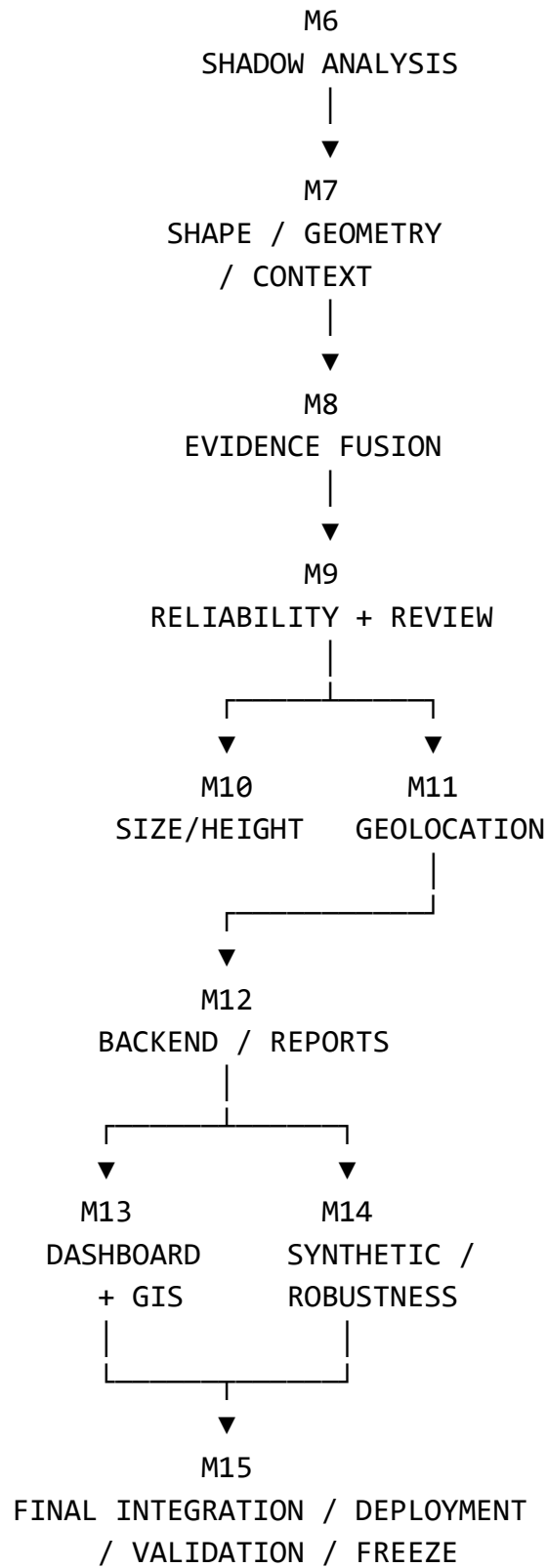
- M9 – Reliability + Unknown Review
- M10 – Physical Estimation
- M11 – Geolocation + Uncertainty
- M12 – Backend / API / Reporting
- M13 – Dashboard / GIS
- M14 – Synthetic Data + Domain Robustness
- M15 – Deployment + Final Integration + Validation

These milestones are **not 16 isolated projects**.

They form one dependency chain.

2. MASTER DEPENDENCY GRAPH





Notice something important:

Some milestones can happen in parallel.

For example:

Member 1 → M2/M3/M4

Member 2 → M5/M6/M7

Member 3 → M12 preparation

Member 4 → M13 preparation

We are not waiting for one person to finish the entire project before everyone else begins.

3. M0 — PROJECT FOUNDATION

Objective

Create a development environment in which all six members can work without constantly breaking each other's code.

Owner

Member 3

Supporting

All members.

What must be created

```
sonar-shield/  
├─ data/  
├─ models/  
└─ detection/
```

- |— segmentation/
- |— preprocessing/
- |— evidence/
- |— geolocation/
- |— backend/
- |— frontend/
- |— synthetic_data/
- |— evaluation/
- |— reports/
- |— docs/

Git structure

At minimum:

- main
- member1-ai
- member2-sonar
- member3-backend
- member4-ui

Module contracts

The following functions/interfaces are defined:

`preprocess(image)`

`detect(image)`

`segment(image)`

`analyze_candidate(image, detection)`

`calculate_reliability(evidence)`

`estimate_size(...)`

`estimate_height(...)`

`geolocate(...)`

`generate_report(...)`

Completion criteria

M0 is complete when:

- everybody can clone the repository;
- everyone can run their environment;
- branches exist;
- module boundaries are documented;
- the backend health endpoint works;
- the project folder structure is stable.

Deliverables

Repository

README

CONTRIBUTING.md

MODULE_CONTRACTS.md

environment files

4. M1 — DATASET + DATA GOVERNANCE

Objective

Create a trustworthy foundation for all AI experiments.

This milestone is broader than simply downloading datasets.

Owner

Member 1

Support

Member 5.

M1.1 — Dataset inventory

Member 5 provides:

Dataset	Classes	Annotation	Source	License	Domain
---------	---------	------------	--------	---------	--------

M1.2 — Class manifest

Member 1 creates:

CLASS_MANIFEST.md

Example:

```
crab_pot
maybe_crab_pot
pipeline
shipwreck
...
```

Only classes with verified labels are included.

M1.3 — Task mapping

Each dataset is assigned to a task:

Detection
Segmentation
Domain robustness
Negative/background
Research-only

M1.4 — Train/validation/test governance

Create:

TRAIN
VALIDATION
ERROR-MINING
FINAL TEST

The **final test set is locked**.

Hard negatives must not be mined from the final test set.

This is a key methodological improvement over a casual train/test workflow.

M1.5 — Data validation

Check:

- broken files
- duplicate files
- missing labels
- incorrect class IDs
- suspicious splits

- near-duplicate survey frames
- image/label correspondence

Completion criteria

M1 is complete when:

Dataset

+

Class manifest

+

Train/Val split

+

Error-mining pool

+

Locked final test

all exist and are reproducible.

Deliverables

datasets/

CLASS_MANIFEST.md

DATASET_REGISTRY.csv

dataset.yaml

split_manifest.csv

5. M2 — BASELINE DETECTION MODEL

Objective

Train the **first real SSS object detector**.

Owner

Member 1

M2.1 — Baseline model selection

Select one initial detector architecture.

The goal is:

one working baseline, not five competing architectures.

M2.2 — Training pipeline

```
dataset
↓
loader
↓
augmentation
↓
training
↓
validation
↓
checkpoint
```

M2.3 — Baseline inference

Input:

SSS image

Output:

```
{  
  "class": "...",  
  "confidence": 0.87,  
  "bbox": [...]  
}
```

M2.4 — Baseline metrics

Measure:

- Precision
- Recall
- F1
- [mAP@0.5](#)
- [mAP@0.5:0.95](#)
- latency/FPS

Completion criteria

M2 is complete when:

An unseen sonar image can pass through the detector and generate a reproducible prediction.

Deliverables

```
model_v1.pt  
train.py  
evaluate.py  
predict.py  
metrics_v1.json  
sample_predictions/
```

6. M3 — AI IMPROVEMENT + HARD-NEGATIVE LEARNING

Objective

Take Model V1 and make it more robust to confusing sonar patterns.

This remains entirely under **Member 1**, as agreed in Stage 2.

M3.1 — Error mining

Run Model V1 on the dedicated **error-mining pool**.

Find:

false positives
false negatives
ambiguous cases
small targets
weak targets

M3.2 — Hard-negative verification

Candidate hard negative:

Model says:
crab pot

Actual:
no target

Member 1 verifies that the image really contains no target.

M3.3 — Difficult positive collection

Examples:

small target
weak acoustic return
partially hidden target

M3.4 — Dataset V2

Create:

Original positives
+
difficult positives
+
hard negatives

M3.5 — Fine-tune

model_v1.pt
↓
dataset_v2
↓
model_v2.pt

M3.6 — Compare V1 vs V2

Use the **same untouched final test set**.

Also use a separate negative benchmark.

Completion criteria

M3 is complete when:

- V1 and V2 have comparable evaluation;
- false-positive behavior is measured;
- the team can explain what errors the hard-negative stage addressed;
- the chosen model is backed by metrics.

Deliverables

```
model_v2.pt  
hard_negatives/  
difficult_positives/  
metrics_v2.json  
v1_vs_v2.csv  
error_analysis.csv
```

7. M4 — SEGMENTATION BRANCH

Objective

Provide pixel-level object masks where suitable labelled masks are available.

Owner

Member 1

M4.1 — Segmentation dataset

Use a dataset with actual pixel-level masks.

M4.2 — Training

```
image
+
mask
↓
segmentation model
↓
predicted mask
```

M4.3 — Metrics

At minimum:

- IoU
- Dice where appropriate
- visual mask inspection

M4.4 — Integration

The output becomes:

mask

which can later support:

- shape analysis
- visualization
- object extent estimation

Completion criteria

M4 is complete when:

A supported segmentation dataset can produce a reproducible pixel mask.

Deliverables

segmentation_model.pt
segmentation inference script
mask examples
segmentation_metrics.json

8. M5 — SONAR PREPROCESSING

Objective

Create a reusable preprocessing pipeline that can transform raw/unaltered SSS imagery before inference.

Owner

Member 2

M5.1 — Normalization

Raw



Intensity normalization

M5.2 — Denoising

Test candidate methods.

M5.3 — CLAHE

Add optional local-contrast enhancement.

M5.4 — Dropout handling

Identify missing/unreliable regions.

M5.5 — Preprocessing experiment

Compare:

RAW
vs
NORMALIZED
vs
FULL PREPROCESS

using detector performance.

Completion criteria

M5 is complete when:

- preprocessing is reproducible;
- it handles supported input resolutions;
- it does not destroy essential sonar structure;
- the team has measured whether it improves or hurts detection.

Deliverables

preprocess.py
normalization.py
denoise.py
clahe.py
preprocessing_config.yaml
preprocessing_experiments.csv

9. M6 — ACOUSTIC SHADOW ANALYSIS

Objective

Extract shadow-related evidence from detected candidates.

Owner

Member 2

M6.1 — Candidate ROI expansion

YOLO bbox



expanded ROI

M6.2 — Shadow candidate extraction

Potential operations:

intensity analysis
thresholding
morphology
connected components
contours

M6.3 — Shadow features

Calculate:

shadow detected?
shadow length
shadow direction
shadow area
shadow/object ratio
shadow strength

M6.4 — Geometry consistency

Use available sonar geometry to determine whether the shadow orientation is at least plausible.

Completion criteria

M6 is complete when:

Given an object bounding box, the system can return a reproducible set of shadow features.

Deliverables

shadow.py
shadow_features.json
visual_shadow_examples/

10. M7 — SHAPE + GEOMETRY + CONTEXT

Objective

Turn the candidate region into additional sonar evidence.

Owner

Member 2

M7.1 — Shape

Extract:

- width
- height
- area
- perimeter
- aspect ratio
- orientation
- compactness
- elongation

M7.2 — Context

Calculate local:

- intensity
- variance
- texture/statistics
- contrast

M7.3 — Geometry

Measure relationships such as:

object ↔ shadow

object ↔ local seabed

object dimensions ↔ shadow dimensions

Completion criteria

M7 is complete when:

Every detection can be converted into a structured feature vector.

Example:

```
{  
  "width_px": 42,  
  "height_px": 20,  
  "aspect_ratio": 2.1,  
  "shadow_ratio": 1.4,  
  "local_variance": 0.18  
}
```

Deliverables

shape.py
geometry.py
context.py
candidate_features.json

11. M8 — EVIDENCE FUSION

Objective

Combine model and sonar evidence.

Owner

Member 2

M8.1 — Evidence schema

Example:

```
{  
  "ai_confidence": 0.84,  
  "shadow_score": 0.76,  
  "shape_score": 0.81,  
  "context_score": 0.68,  
  "geometry_score": 0.73  
}
```

M8.2 — Initial fusion model

Start with an interpretable engineering baseline.

Example:

$$\begin{aligned} R = & \\ & w1 \cdot AI \\ & + \\ & w2 \cdot Shadow \\ & + \\ & w3 \cdot Shape \\ & + \\ & w4 \cdot Context \\ & + \\ & w5 \cdot Geometry \end{aligned}$$

The exact weights must be validated.

M8.3 — Weight/threshold experiment

Use the validation set to test reasonable alternatives.

Completion criteria

M8 is complete when:

Given a detection and evidence features, the system returns one reproducible Reliability Index.

Deliverables

fusion.py
fusion_config.yaml
fusion_results.csv

12. M9 — RELIABILITY + UNKNOWN REVIEW

Objective

Turn evidence into an operational decision.

Owner

Member 2

M9.1 — Reliability Index

Output:

0.00 → 1.00

but it should be named:

Reliability Index

unless you later calibrate it.

M9.2 — Decision states

CONFIRM

REVIEW

REJECT

M9.3 — Unknown/low-confidence pathway

If the candidate doesn't confidently fit a trained category:

UNKNOWN / REVIEW REQUIRED

M9.4 — Explainability

For every review result, retain:

Why did the system assign this state?

For example:

Detector confidence: low

Shadow evidence: weak

Shape evidence: inconsistent

→ REVIEW

Completion criteria

M9 is complete when:

Every candidate gets both a Reliability Index and a decision state.

Deliverables

```
reliability.py  
decision.py  
decision_thresholds.yaml  
evidence_audit.json
```

13. M10 — SIZE + HEIGHT ESTIMATION

Objective

Estimate physical dimensions only when the available sonar information supports doing so.

Owner

Member 2

M10.1 — Size

```
pixel dimensions  
↓  
sonar scale
```

↓
physical dimensions

M10.2 — Height

Potentially:

shadow length
+
sonar geometry
↓
approximate height

M10.3 — Uncertainty

Every estimate gets:

estimated
+
uncertainty

or:

unavailable

Completion criteria

M10 is complete when:

- dimensions can be calculated for supported calibrated inputs;
- unsupported inputs return unavailable;
- height does not get fabricated.

Deliverables

size.py
height.py
measurement_uncertainty.py

14. M11 — GEOLOCATION + UNCERTAINTY

Objective

Transform supported sonar detections into geographic positions.

Owner

Member 3

Supporting information

Member 2 supplies sonar geometry/features.

M11.1 — Metadata parser

Read:

GPS
heading
range
timestamp
sensor offsets
vehicle pose

M11.2 — Sonar coordinate transformation

image coordinate
↓
sonar-relative coordinate

M11.3 — Geographic transformation

sonar coordinate
+
navigation
↓
latitude / longitude

M11.4 — Location uncertainty

Return:

location
+
uncertainty

M11.5 — Missing metadata

If insufficient:

location_status = unavailable

Completion criteria

M11 is complete when:

- supported metadata generates reproducible geographic output;
- missing metadata does not crash the system;
- uncertainty is represented;
- no coordinates are invented.

Deliverables

metadata_parser.py

geolocation.py

uncertainty.py

15. M12 — BACKEND + API + REPORTING

Objective

Expose the complete pipeline as a usable software service.

Owner

Member 3

M12.1 — Health endpoint

GET /health

M12.2 — Detection endpoint

POST /detect

M12.3 — Full analysis endpoint

POST /analyze

Flow:

image
↓
preprocess
↓
AI
↓
evidence
↓
reliability
↓
geolocation
↓
result

M12.4 — Report endpoint

POST /report

M12.5 — JSON report

Structured output.

M12.6 — CSV report

Tabular output.

M12.7 — Error handling

Handle:

- invalid file
- missing metadata
- model unavailable
- no detection
- unsupported input
- processing failure

Completion criteria

M12 is complete when:

A client can submit an image to `/analyze` and receive a complete structured SONAR-SHIELD response.

Deliverables

FastAPI application

API schemas

API documentation

sample JSON

sample CSV

16. M13 — DASHBOARD + GIS

Objective

Make the product usable by a human.

Owner

Member 4

Integration

Member 3.

M13.1 — Upload

M13.2 — Original/processed image

M13.3 — Detection overlay

Show:

class
confidence
bbox

M13.4 — Segmentation overlay

Where applicable.

M13.5 — Evidence panel

AI
Shadow
Shape
Context
Geometry

M13.6 — Reliability panel

Reliability Index
Decision

M13.7 — Map

Display:

lat/lon
object marker
uncertainty

when available.

M13.8 — Reports

Download:

JSON

CSV

Completion criteria

M13 is complete when:

A non-developer can upload a supported sonar image and understand the resulting detection without looking at the source code.

Deliverables

frontend/

dashboard/

map component

report download UI

17. M14 — SYNTHETIC DATA + DOMAIN ROBUSTNESS

This milestone has **two related but distinct goals**.

PART A — Synthetic data

Owner

Member 4

Reviewer

Member 1

M14.1 — Blender scenes

Generate objects such as:

- pipe
- cylinder
- debris
- structures
- net-like forms where practical

M14.2 — Synthetic sonar-like images

Produce grayscale/noisy/shadowed examples.

M14.3 — Labels

Create bounding boxes or other supported labels.

M14.4 — Quality check

Member 1 decides:

Is this data useful enough to enter training?

Completion criteria

M14 synthetic-data portion is complete when:

- images exist;
- labels exist;
- quality has been checked;
- the data is clearly marked synthetic;
- synthetic data isn't used as the final real-world test benchmark.

PART B — Domain robustness

Owner

Member 1

Support

Member 2.

Evaluate:

different sonar
different survey
different site
different resolution
different intensity distribution

where datasets permit.

Completion criteria

The team has documented:

Where the model generalizes and where it fails.

This is more valuable than claiming universal robustness.

18. M15 — FINAL INTEGRATION + DEPLOYMENT + VALIDATION

Objective

Bring everything together.

M15.1 — Final model

Member 1 provides:

```
final_model.pt  
final_model.onnx  
segmentation_model
```

M15.2 — Final sonar intelligence

Member 2 provides:

```
preprocessing  
shadow  
shape
```

context
fusion
reliability
size
height

M15.3 — Final geographic/reporting layer

Member 3 connects:

geolocation
uncertainty
JSON
CSV

M15.4 — Final dashboard

Member 4 connects to the production API.

M15.5 — End-to-end pipeline

The system must execute:

UPLOAD
↓
VALIDATE
↓
PREPROCESS
↓
DETECT
↓
SEGMENT where applicable

↓
EVIDENCE
↓
RELIABILITY
↓
DECISION
↓
SIZE / HEIGHT where available
↓
GEOLOCATION where available
↓
UNCERTAINTY
↓
MAP
↓
JSON / CSV

M15.6 — Deployment

Member 3 packages the system using Docker.

M15.7 — ONNX verification

Member 1 checks the exported model.

M15.8 — TensorRT benchmark

Only if practical and supported by available hardware.

Measure:

- latency
- FPS
- memory

M15.9 — Final test

Use the **locked final test set**.

Do not modify it after results are obtained.

M15.10 — Failure analysis

Record:

false positives
false negatives
unknowns
metadata failures
preprocessing failures
geolocation failures

M15.11 — Ablation

Compare the important system stages.

Example:

A — Baseline AI
B — + Preprocessing
C — + Hard Negatives
D — + Evidence
E — Full system

Completion criteria

M15 is complete when:

The entire system can process a representative supported input from upload through final report, and every major result can be traced back to an actual implementation rather than placeholder data.

Deliverables

final_model.pt
final_model.onnx
segmentation_model
Docker configuration
working backend
working dashboard
test report
ablation report
failure-analysis report
final JSON/CSV examples

19. MILESTONE EXIT-GATE SYSTEM

This is important.

We don't move to the next major stage simply because:

"The code exists."

Each milestone has an **exit gate**.

M0 exit gate

Everyone can run the project.

M1 exit gate

Dataset is reproducible

+

classes verified

+

final test locked.

M2 exit gate

SSS image → Model V1 → detection.

M3 exit gate

V1 vs V2 evaluated

+

hard-negative effect measured.

M4 exit gate

Image → pixel mask.

M5 exit gate

Raw → reproducibly processed.

M6 exit gate

Detection → shadow features.

M7 exit gate

Detection → structured sonar features.

M8 exit gate

Evidence → Reliability Index.

M9 exit gate

Reliability → Confirm/Review/Reject.

M10 exit gate

Dimensions produced only when supported.

M11 exit gate

Valid metadata → geographic result.

Missing metadata → graceful unavailable state.

M12 exit gate

API → complete structured result.

M13 exit gate

Human can use the application.

M14 exit gate

Synthetic/domain experiments documented.

M15 exit gate

Complete end-to-end reproducible system.

20. CRITICAL DEPENDENCIES

This table tells the team **what must exist before something else can really work**.

Milestone	Depends on
M0	Nothing
M1	M0
M2	M1
M3	M2 + M1
M4	M1
M5	M0/M1 sample images
M6	M5 + M2 detections
M7	M5 + M6
M8	M2 + M6 + M7

M9	M8
M10	M2 + M7 + sonar scale
M11	M2 + metadata + geometry
M12	M2 + M8 + M9 + M11
M13	M12
M14	M1 + M4
M15	M2–M14 relevant outputs

The important thing is that **M5–M7 can happen while M2/M3 are still being developed.**

That is how you get parallel development rather than waiting.

21. WHAT CAN BE DEVELOPED IN PARALLEL?

This is especially important for your September 26–29 window.

Member 1 can work on:

M1
M2
M3
M4

while...

Member 2 works on:

M5
M6 prototypes
M7 prototypes

while...

Member 3 works on:

M0

M12 skeleton

M11 metadata/geolocation framework

while...

Member 4 works on:

M13 UI

M14 synthetic data

while...

Member 5 works on:

M1 support

M14 data verification

while...

Member 6 works on:

documentation

QA

experiment tracking

This is the reason the team can attempt the complete feature set within the compressed period.

22. CRITICAL HANDOFFS BETWEEN MILESTONES

Handoff A — Member 1 → Member 2's context

After M2:

model_v1
+
prediction format

Member 2 uses the detector output to develop evidence analysis.

Handoff B — Member 1 → Member 1

M3 remains under Member 1:

V1
↓
hard negatives
↓
V2

No cross-team training dependency here.

Handoff C — Member 2 → Member 3

After M9:

evidence
+
reliability

+
decision

Handoff D — Member 2 → Member 3

After M10:

size
+
height

Handoff E — Member 3 → Member 4

After M12:

API
+
JSON schema

Member 4 can now connect the real dashboard.

Handoff F — Member 1 + Member 2 + Member 3 → Final

AI
+
Sonar Intelligence
+
Backend / Geospatial

becomes:

complete application

23. WHICH FEATURES ARE ESSENTIAL FOR THE FINAL PRODUCT?

Even though we're attempting the full feature list, these are the **architectural core**:

1. Preprocessing
2. Detection
3. Shadow analysis
4. Shape/context evidence
5. Evidence fusion
6. Reliability
7. Review pathway
8. Reporting
9. Dashboard

Without those, SONAR-SHIELD loses its central identity.

The other components strengthen the system.

24. THE RESEARCH CONTRIBUTION MILESTONES

From a research perspective, these are the most important:

Research contribution 1

Baseline detection

Can AI identify sonar targets?

Research contribution 2

Hard-negative learning

Can difficult sonar backgrounds reduce false positives?

Research contribution 3

Shadow-aware evidence

Does adding acoustic-shadow evidence help distinguish candidates?

Research contribution 4

Reliability

Can multiple evidence sources provide a more informative decision than raw detector confidence?

Research contribution 5

Domain robustness

How does the system behave across different sonar/data conditions?

These should be reflected in your experiment logs.

25. THE MOST IMPORTANT MILESTONES FOR YOUR NOVELTY

If a judge asks:

"Where is your innovation?"

the milestone chain should be:

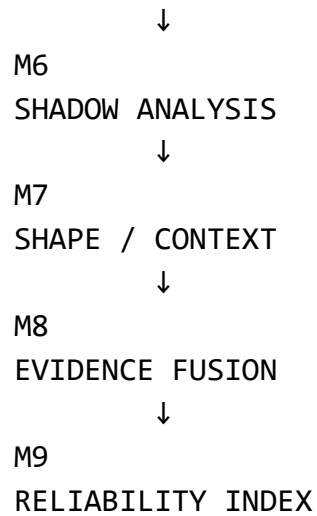
M2

BASELINE YOLO



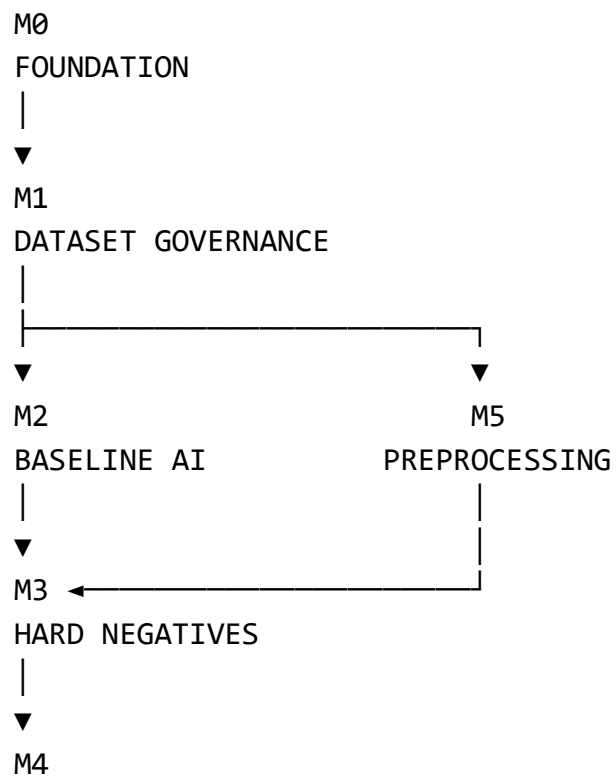
M3

HARD-NEGATIVE IMPROVEMENT



That is the heart of SONAR-SHIELD.

26. THE COMPLETE TECHNICAL DEVELOPMENT CHAIN



SEGMENTATION

M5 + M2



M6

SHADOW



M7

SHAPE + CONTEXT + GEOMETRY



M8

EVIDENCE FUSION



M9

RELIABILITY / REVIEW

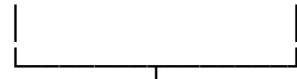


M10

SIZE/HEIGHT

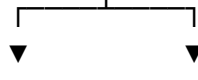
M11

GEOLOCATION



M12

API / REPORTING

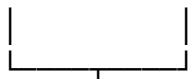


M13

UI

M14

SYNTHETIC /
ROBUSTNESS



27. STAGE 4 FINAL DEFINITION

So now we have:

M0

Project foundation

M1

Dataset & data governance

M2

Baseline detection

M3

Hard-negative learning / AI improvement

M4

Segmentation

M5

Preprocessing

M6

Shadow analysis

M7

Shape + geometry + context

M8

Evidence fusion

M9

Reliability + unknown/review

M10

Size + height

M11

Geolocation + uncertainty

M12

Backend + API + reporting

M13

Dashboard + GIS

M14

Synthetic data + domain robustness

M15

Final integration + deployment + validation

STAGE 4 CHECKPOINT

At this point the project has four clearly separated layers:

WHAT WE ARE BUILDING



STAGE 1

WHO BUILDS IT



STAGE 2

WHAT FEATURES EXIST



STAGE 3

WHAT "DONE" MEANS



STAGE 4

The next stage is the one that will turn all of this into your **actual September 26 →**

September 30 execution plan: each day's milestones, which member works on which milestone, what they should finish before the next handoff, what can happen in parallel, what must be frozen on September 29, and what remains for the September 30 submission morning.

STAGE 5 — FINAL DAY-BY-DAY EXECUTION PLAN

This is the **final operational plan**: what every member does on every day from **September 26 through September 30**, what must be delivered, what can happen in parallel, what depends on another person's work, and what must be frozen before submission.

Your original team document establishes Members 1–3 as the heavy technical core, Member 4 as UI/synthetic-data, Member 5 as data/research, and Member 6 as documentation/PPT/demo.

We are using that structure, but with the revised ownership you approved:

MEMBER 1

AI + ALL TRAINING

+ HARD NEGATIVES

+ SEGMENTATION

+ MODEL EVALUATION

MEMBER 2

SONAR INTELLIGENCE

+ PREPROCESSING

+ SHADOW

+ SHAPE

+ CONTEXT

+ EVIDENCE

+ RELIABILITY

MEMBER 3

BACKEND

+ GEOLOCATION

+ TRACKING

+ REPORTING

+ INTEGRATION

+ DEPLOYMENT

MEMBER 4

UI

+ SYNTHETIC DATA

MEMBER 5

DATA + RESEARCH

MEMBER 6

DOCUMENTATION + QA + DEMO

OVERALL DEADLINE STRUCTURE

September 26 — DAY 1

Foundation + parallel development + training begins

September 27 — DAY 2

Model V1 + preprocessing/evidence modules + first major handoff

September 28 — DAY 3

Model V2 + evidence fusion + reliability + backend integration

September 29 — DAY 4

Final AI + geolocation + dashboard + complete integration + freeze

September 30 — DAY 5

Final verification + submission

The original plan envisioned a longer development cycle from datasets and baseline modeling through integration, testing and submission; this schedule compresses those stages into your remaining four development days.

BEFORE WE START — ONE CRITICAL RULE

The final test set is locked from Day 1.

Your AI workflow is:

TRAIN
↓
MODEL V1
↓
ERROR-MINING POOL
↓
Hard negatives
↓
MODEL V2
↓
VALIDATION
↓
FINAL TEST ← untouched

Do **not** take hard negatives from the final test set.

That is essential for credible evaluation.

SEPTEMBER 26 — DAY 1

MILESTONE: FOUNDATION + ALL WORKSTREAMS START

Day 1 success condition

By the end of September 26:

Repository 
Datasets prepared 
AI training running 
Preprocessing prototype 
Evidence prototype 
Backend skeleton 
UI skeleton 

Research/data inventory 

Documentation framework 

Nobody should be waiting for someone else at the end of Day 1.

MEMBER 1 — AI/ML

Main goal

Get the baseline training pipeline running.

Task 1 — Environment

Set up:

- Python
- PyTorch
- Ultralytics
- CUDA/GPU
- OpenCV
- NumPy

Verify the GPU works.

Task 2 — Dataset preparation

Create:

```
datasets/  
└─ baseline/  
    │   └─ images/  
    │       └─ train/  
    │       └─ val/  
    └─ test/
```

```
└─ labels/
   └─ train/
   └─ val/
   └─ test/
```

Task 3 — Class manifest

Create:

`CLASS_MANIFEST.md`

Every class must have:

- actual labelled data
- known annotation type
- dataset source
- intended task

No invented classes.

Task 4 — Lock the final test set

Create:

`final_test/`

and immediately mark it:

DO NOT USE FOR TRAINING OR ERROR MINING

Task 5 — Training script

Create:

```
train_v1.py  
evaluate.py  
predict.py  
dataset.yaml
```

Task 6 — Start Model V1 training

Start the baseline model.

The goal is not perfection yet.

The goal is:

Get the first real sonar detector training successfully.

Day 1 deliverables

```
dataset.yaml  
train_v1.py  
predict.py  
CLASS_MANIFEST.md  
locked final-test split  
baseline training run
```

MEMBER 2 — SONAR INTELLIGENCE

Member 2 does **not** wait for Model V1.

The original plan already assigns this role preprocessing, confidence/filtering and geotagging.

Main goal

Build the entire sonar-processing skeleton independently.

Task 1 — Preprocessing

Create:

```
preprocessing/  
├─ normalize.py  
├─ denoise.py  
├─ clahe.py  
└─ pipeline.py
```

Pipeline:

```
RAW  
↓  
NORMALIZATION  
↓  
DENOISING  
↓  
CLAHE  
↓  
OUTPUT
```

Task 2 — Shadow prototype

Create:

```
evidence/  
└─ shadow.py
```

For now, give it a manually supplied box:

`image + bbox`

and make it return:

```
shadow_detected  
shadow_length  
shadow_direction  
shadow_score
```

Task 3 — Shape prototype

Create:

```
shape.py
```

Return:

- width
- height
- area
- aspect ratio
- orientation

Task 4 — Context prototype

Create:

```
context.py
```

Return things such as:

- local intensity
- variance
- simple texture/context features

Task 5 — Reliability framework

Create the initial structure:

AI score
Shadow score
Shape score
Context score
Geometry score

No need to finalize weights yet.

Task 6 — Geolocation framework

Create the metadata-handling skeleton:

GPS
heading
range
timestamp
sensor offset

Day 1 deliverables

preprocessing/
shadow.py
shape.py
context.py
geolocation skeleton
evidence schema

MEMBER 3 — SYSTEM / BACKEND

Main goal

Build the software skeleton before the AI is ready.

Task 1 — Repository architecture

Create the agreed folder structure.

Task 2 — FastAPI

Implement:

GET /health
POST /detect
POST /analyze
POST /report

Initially /detect can return mock JSON.

Task 3 — Module contracts

Define the exact interfaces:

`preprocess(image)`

`detect(image)`

`analyze_candidate(image, detection)`

```
calculate_reliability(evidence)
```

```
geolocate(detection, metadata)
```

```
generate_report(result)
```

Task 4 — Error handling foundation

Set up basic handling for:

- invalid file
- unsupported file
- missing model
- missing metadata

Task 5 — Docker skeleton

Create the initial Docker configuration.

Don't optimize it yet.

Day 1 deliverables

FastAPI skeleton

module contracts

health endpoint

backend structure

Docker skeleton

MEMBER 4 — UI + SYNTHETIC DATA

UI

Build:

Upload
Run Analysis
Image viewer
Results panel
Map placeholder
Report download

Use mock JSON if necessary.

Synthetic

Set up Blender and create the first simple underwater scene:

seabed
+
pipe
+
cylinder
+
debris

Don't worry about a huge synthetic dataset yet.

Deliverables

working UI skeleton
first synthetic scene

MEMBER 5 — DATA + RESEARCH

Main goal

Create the project data registry.

For every dataset:

Name

Source

License

Images

Classes

Annotation type

Sensor/domain

Also check that the classes claimed by the team actually exist.

Deliverable

DATASET_REGISTRY.csv

MEMBER 6 — DOCUMENTATION + QA

Create:

README.md

EXPERIMENT_LOG.md

TEST_CHECKLIST.md

Prepare a results table template.

DAY 1 EXIT GATE

By the end of September 26:

DAY 1

M1 → baseline training running
M2 → preprocessing/evidence prototypes
M3 → backend skeleton
M4 → UI skeleton
M5 → dataset registry
M6 → documentation/QA framework

No handoff is required yet.

Everyone is building independently.

SEPTEMBER 27 — DAY 2

MILESTONE: MODEL V1 + SONAR PIPELINE READY

This is the **first major dependency day**.

The most important event:

Member 1 finishes Model V1 and hands its outputs into the rest of the pipeline.

MEMBER 1

Main goal

Finish Model V1.

Task 1 — Finish training

Training:

Dataset



Model



Validation



Checkpoint

Task 2 — Evaluate

Measure:

- Precision
- Recall
- F1
- mAP50
- mAP50-95
- inference speed

Task 3 — Run prediction

Run V1 on:

Error-mining pool

Not final test.

Save:

predictions/v1/

Task 4 — Error analysis

Classify:

False positives

False negatives

Low-confidence cases

Example:

rock → crab pot

shadow → debris

real target → missed

Task 5 — Package V1

Create:

```
models/v1/  
├─ model_v1.pt  
├─ metrics.json  
└─ classes.yaml
```



MAJOR HANDOFF — MEMBER 1

Member 1 gives:

```
model_v1.pt  
+  
V1 predictions  
+  
V1 metrics  
+  
error_analysis.csv
```

to the project pipeline.

Important

Member 1 **does not hand over AI ownership.**

Member 1 continues to own all future AI training.

MEMBER 2

While Member 1 is training, Member 2 finishes their independent work.

Main goal

Make the sonar pipeline ready to consume a YOLO bounding box.

Finish:

```
preprocess(image)  
shadow_analysis(image, bbox)
```

```
shape_analysis(image, bbox)
context_analysis(image, bbox)
```

Test them manually.

Build evidence output

Example:

```
{
  "shadow_score": 0.72,
  "shape_score": 0.81,
  "context_score": 0.68
}
```

Prepare Reliability Index

Create the schema:

AI
Shadow
Shape
Context
Geometry

Don't over-tune the formula yet.

MEMBER 3

Main goal

Replace the mock AI call with the real Model V1.

Pipeline:

Upload



Preprocess



Model V1



Bounding box



Evidence Engine

Create:

POST /analyze

MEMBER 4

Main goal

Display the real Model V1 output.

Show:

- bounding box
- class
- detector confidence
- processed image

Keep the UI functional, not decorative.

MEMBER 5

Main goal

Help Member 1 with:

- label verification
- difficult examples
- dataset issues
- error-mining examples

MEMBER 6

Document:

Model V1

Metrics

Failure examples

Screenshots

DAY 2 EXIT GATE

The system should now support:

SSS image

↓

preprocessing

↓

Model V1

↓
bounding box
↓
initial sonar evidence

Critical deliverable

MODEL V1

is versioned and reproducible.

SEPTEMBER 28 — DAY 3

MILESTONE: MODEL IMPROVEMENT + EVIDENCE FUSION + RELIABILITY + CORE BACKEND

This is the biggest research/engineering day.

MEMBER 1 — MAIN TASK

Hard-negative learning

Member 1 owns the entire loop.

Step 1

Take V1 errors from the error-mining pool.

Step 2

Identify valid hard negatives:

rock
shadow
seabed
noise
reverberation

Step 3

Combine:

Original positives
+
Difficult positives
+
Hard negatives

Step 4

Train:

model_v1.pt
↓
dataset_v2
↓
model_v2.pt

Step 5 — Compare V1 and V2

Use the **same locked final test set** only for final comparison, not for building V2.

Also evaluate on the negative benchmark.

Record:

Precision
Recall
F1
mAP50
mAP50-95
False positives
Inference speed

Step 6 — Segmentation

While V2 is training/evaluating, Member 1 also works on the segmentation branch:

SSS image
↓
segmentation
↓
mask

using an appropriately labelled mask dataset.

Step 7 — Augmentation

Experiment with relevant sonar variations.

MEMBER 2 — MAIN TASK

Member 2 is **not doing hard-negative training**.

They now focus entirely on SONAR-SHIELD intelligence.

Task 1 — Shadow engine

Use the real Model V1/V2 boxes.

Calculate:

shadow detected
shadow length
shadow direction
shadow score
shadow/object relationship

Task 2 — Shape

Calculate:

width
height
area
aspect ratio
orientation
compactness

Task 3 — Context

Calculate:

local intensity
variance
texture
contrast

Task 4 — Geometry

Create the object/shadow geometric features.

Task 5 — Evidence fusion

Input:

AI
Shadow
Shape
Context
Geometry

Output:

Evidence Vector

Task 6 — Reliability Index

Implement:

Evidence
↓
Reliability Index

Task 7 — Decision

Implement:

CONFIRM
REVIEW
REJECT

Task 8 — Unknown path

Implement:

UNKNOWN / LOW-CONFIDENCE
HUMAN REVIEW REQUIRED

MEMBER 3 — MAIN TASK

Integrate:

Model V1/V2
+
Preprocessing
+
Evidence
+
Reliability

Task 1 — Full analysis API

POST /analyze

should return something like:

```
{
  "detections": [
    {
      "class": "crab_pot",
      "detector_confidence": 0.84,
      "shadow_score": 0.76,
      "shape_score": 0.81,
      "context_score": 0.68,
```

```
        "reliability_index": 0.81,  
        "decision": "CONFIRM"  
    }  
]  
}
```

Task 2 — Tracking/persistence integration

Only if sequential sonar data exists.

Task 3 — Geolocation framework

Connect:

```
detection  
+  
metadata
```

to the geographic transformation pipeline.

MEMBER 4

Main task

Build the real evidence dashboard.

Display:

```
Detector confidence  
Shadow evidence  
Shape evidence  
Context
```

Reliability Index
Decision

MEMBER 5

Main task

Verify every hard-negative candidate used by Member 1.

Also identify:

- mislabeled examples
- suspicious labels
- dataset inconsistencies

MEMBER 6

Main task

Create the main research evidence:

V1

↓

Hard negatives

↓

V2

↓

Evidence fusion

↓

Reliability

Prepare screenshots and results tables.

DAY 3 EXIT GATE

At the end of September 28, the team should have:

MODEL V2 

DETECTION 

PREPROCESSING 

SHADOW 

SHAPE 

CONTEXT 

EVIDENCE FUSION 

RELIABILITY 

DECISION 

BACKEND ANALYSIS API 

This is the point where SONAR-SHIELD stops being just a detector.

SEPTEMBER 29 — DAY 4

MILESTONE: FULL PRODUCT INTEGRATION + FINAL VALIDATION + FREEZE

This is the final development day.

Everything that is already built gets connected into one system.

MEMBER 1 — FINAL AI

Task 1 — Select final detector

Compare:

Model V1

vs

Model V2

Use validation/test evidence.

Do not automatically select V2 simply because it is newer.

Task 2 — Final segmentation model

Finalize the best working segmentation version.

Task 3 — Export

Prepare:

`final_model.pt`

`final_model.onnx`

`segmentation_model`

`classes.yaml`

`metrics.json`

Task 4 — AI ablation report

Finalize:

Baseline

+

Preprocessing

+

Hard negatives

+

Evidence

+

Full system

with the metrics that were actually measured.

Task 5 — ONNX verification

Check that ONNX inference is consistent with the original model.

Task 6 — TensorRT

Only if practical.

Benchmark:

- latency
- FPS
- memory

No unsupported performance claims.

MEMBER 2 — FINAL SONAR INTELLIGENCE

Task 1 — Freeze preprocessing

normalize
denoise
CLAHE

Task 2 — Freeze evidence

shadow
shape
context
geometry

Task 3 — Freeze Reliability

Reliability Index
Confirm
Review
Reject
Unknown

Task 4 — Physical estimates

Where supported:

length
width
height

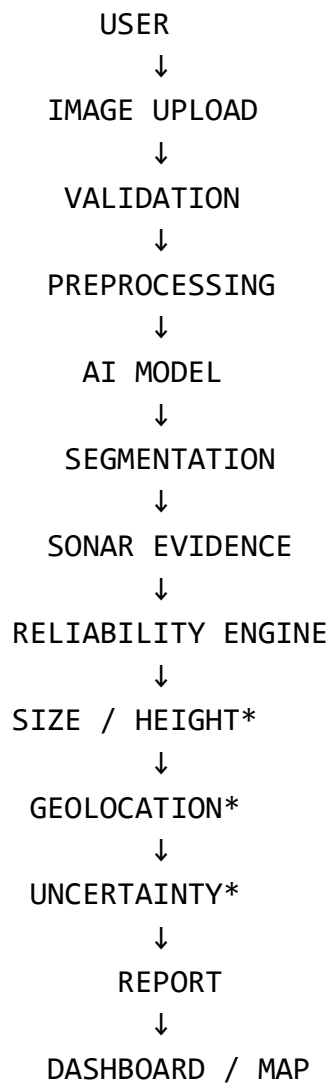
Otherwise:

unavailable

MEMBER 3 — FINAL APPLICATION OWNER

This is Member 3's biggest responsibility.

Integrate:



Task 1 — Geolocation

Finalize:

metadata parser
coordinate conversion
location uncertainty
missing-metadata handling

Task 2 — API

Finalize:

GET /health
POST /detect
POST /analyze
POST /report

Task 3 — Report pipeline

Make:

JSON
CSV

generated from actual results.

Task 4 — Dashboard integration

Connect Member 4's frontend to actual API results.

Task 5 — Docker

Make the complete system reproducible.

Task 6 — Error handling

Test:

bad file
missing metadata
no detection
multiple detection
missing model
API failure

MEMBER 4 — FINAL UI

The UI must now use **real data only**.

Final screen:

UPLOAD
↓
ORIGINAL IMAGE
↓
PROCESSED IMAGE
↓
DETECTION
↓
SEGMENTATION
↓
EVIDENCE
↓
RELIABILITY
↓

DECISION



MAP



REPORT

Ensure:

- map works
- filters work
- report download works
- unknown/review state is visible
- missing location is clearly represented

MEMBER 5 — FINAL DATA/RESEARCH AUDIT

Check:

Dataset source

Classes

Licenses

Annotations

References

Claims

Especially check that nobody is saying:

"Ghost Pot dataset = ghost-net dataset."

unless actual labels support that claim.

MEMBER 6 — FINAL QA/DOCUMENTATION

Compile:

Architecture
Dataset table
Metrics
Ablations
Failure cases
Screenshots
Known limitations
Demo flow
README
Final report

Documentation must describe **implemented features**, not merely planned features.

SEPTEMBER 29 — FINAL SYSTEM FREEZE

This is the most important rule.

At the end of Day 4:

The best working state becomes the final build.

From the freeze onward:

Allowed

- bug fixes
- broken path fixes
- missing dependency fixes
- documentation fixes
- submission packaging

Not allowed

- new model architecture
- new major dataset
- new evidence algorithm
- new UI architecture
- experimental retraining that could break the final model
- major geolocation rewrite

SEPTEMBER 30 — DAY 5

MILESTONE: FINAL VERIFICATION + SUBMISSION

This should be treated as **submission day, not development day**.

MEMBER 1

Verify:

final_model.pt 

ONNX 

segmentation model 

metrics 

class manifest 

Run one final inference.

MEMBER 2

Verify:

preprocessing 
shadow 
shape 
context 
evidence fusion 
reliability 

Test missing/weak evidence.

MEMBER 3

Run the complete system:

UPLOAD
↓
PREPROCESS
↓
DETECT
↓
EVIDENCE
↓
RELIABILITY
↓
GEOLOCATION
↓
REPORT

Verify:

- API
- frontend

- map
- reports
- Docker/package
- error handling

MEMBER 4

Run UI acceptance tests:

Upload 

Detection display 

Evidence 

Decision 

Map 

Download 

MEMBER 5

Verify:

datasets

sources

licenses

references

class descriptions

MEMBER 6

Verify final:

README
documentation
screenshots
demo
PPT/synopsis
submission assets

FINAL SYSTEM ACCEPTANCE CHECKLIST

Before submission, all of these should be explicitly marked:

Input

- ☐ SSS image upload
- ☐ metadata input
- ☐ invalid-input handling

AI

- ☐ detection
- ☐ segmentation where supported
- ☐ model confidence
- ☐ final model version

Learning

- ☐ augmentation
- ☐ hard negatives
- ☐ difficult positives
- ☐ V1 vs V2 evaluation

Sonar intelligence

- ☐ preprocessing
- ☐ shadow

- ☐ shape
- ☐ context
- ☐ geometry
- ☐ evidence fusion

Decision

- ☐ Reliability Index
- ☐ Confirm
- ☐ Review
- ☐ Reject
- ☐ Unknown/review

Physical estimation

- ☐ size where supported
- ☐ height where supported
- ☐ uncertainty
- ☐ unavailable-state handling

Geospatial

- ☐ metadata parsing
- ☐ geolocation
- ☐ uncertainty
- ☐ missing metadata handling

Application

- ☐ FastAPI
- ☐ dashboard
- ☐ map
- ☐ filters
- ☐ JSON
- ☐ CSV

Deployment

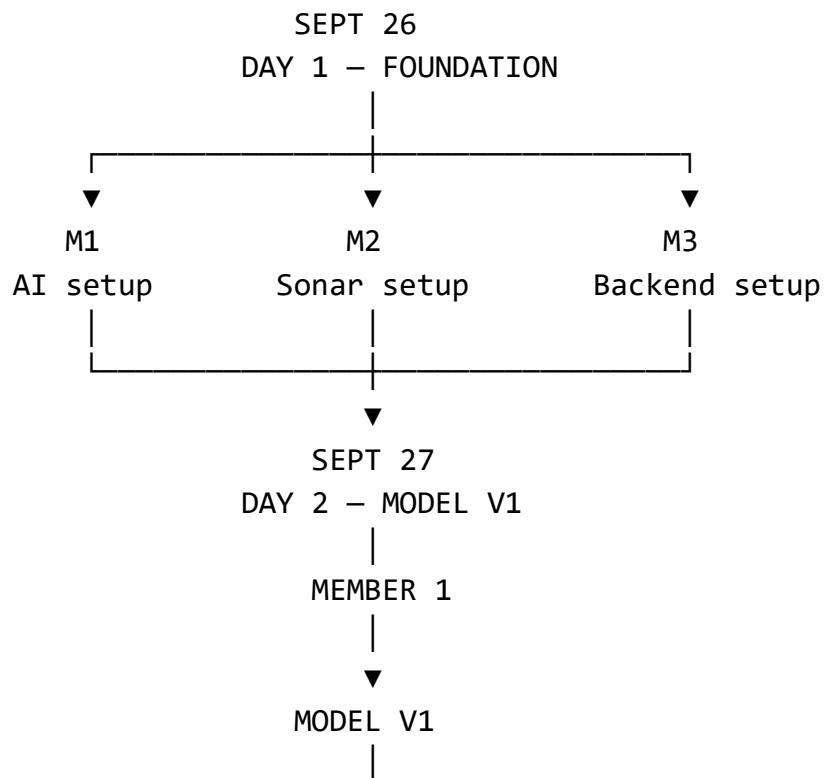
- ☐ ONNX
- ☐ TensorRT benchmark if practical
- ☐ Docker

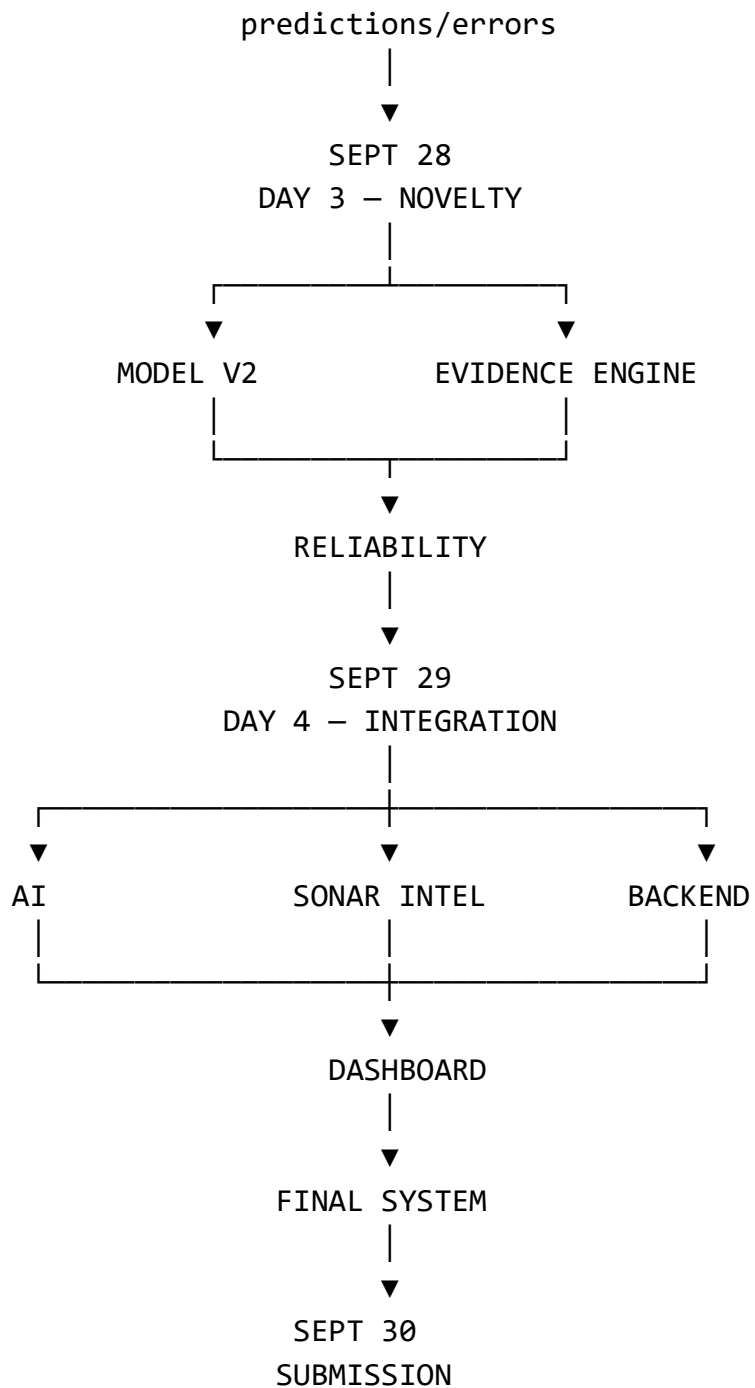
Validation

- ☐ final test set
- ☐ false-positive benchmark
- ☐ ablation
- ☐ failure analysis
- ☐ cross-domain test where possible

THE 4-DAY CRITICAL PATH

This is the part you should send directly to the team.





DAILY MILESTONE SUMMARY

Day	Date	Master milestone	Critical output
-----	------	------------------	-----------------

Day	Sep	Foundation + parallel	Training running +
1	26	pipelines	preprocessing/evidence/backend/UI skeleton
Day	Sep	Baseline AI complete	Model V1 + predictions + metrics
2	27		
Day	Sep	Robust AI + SONAR	Model V2 + evidence fusion + reliability
3	28	intelligence	
Day	Sep	Full product integration	Complete SONAR-SHIELD build frozen
4	29		
Day	Sep	Final	Submission-ready package
5	30	verification/submission	

THE MOST IMPORTANT HANDOFFS

September 27

Member 1

MODEL V1
+
PREDICTIONS
+
METRICS

This allows the real pipeline to consume the detector.

September 28

Member 1

MODEL V2
+
SEGMENTATION
+
AI METRICS

Member 2

PREPROCESSING

+

EVIDENCE

+

RELIABILITY

These go to Member 3.

September 29

Member 1 + Member 2

FINAL AI

+

FINAL SONAR INTELLIGENCE

↓

Member 3

FINAL APPLICATION

↓

Member 4

FINAL USER INTERFACE

↓

Member 5 + Member 6

FINAL DATA / DOCUMENTATION / QA

↓

FINAL SONAR-SHIELD

WHAT EACH MEMBER SHOULD BE ABLE TO SAY AT THE END

Member 1

"I can show how Model V1 was trained, what errors it made, how hard-negative learning produced V2, and how we selected the final model."

Member 2

"I can show how a detector candidate is analyzed using acoustic shadow, shape, geometry and seabed context, and how those features become a Reliability Index."

Member 3

"I can show the complete software flow from upload through AI, evidence, geolocation and report generation."

Member 4

"I can demonstrate the dashboard and explain what every displayed result means."

Member 5

"I can explain every dataset, annotation type, source and license."

Member 6

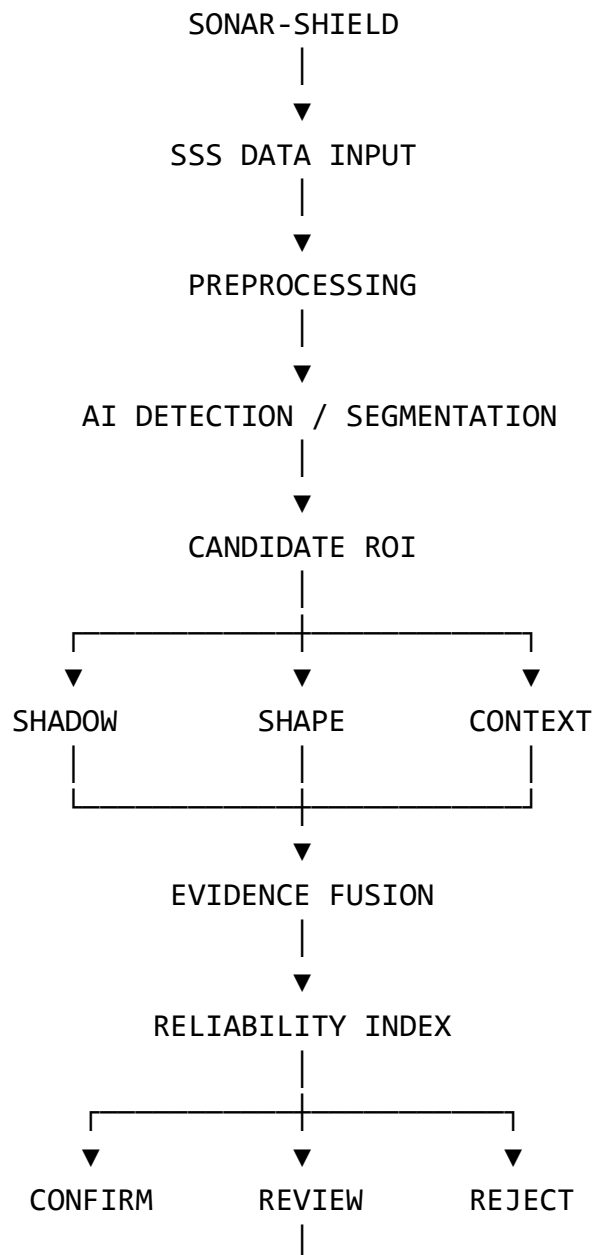
"I can explain the complete project, show the test evidence and demonstrate the final system."

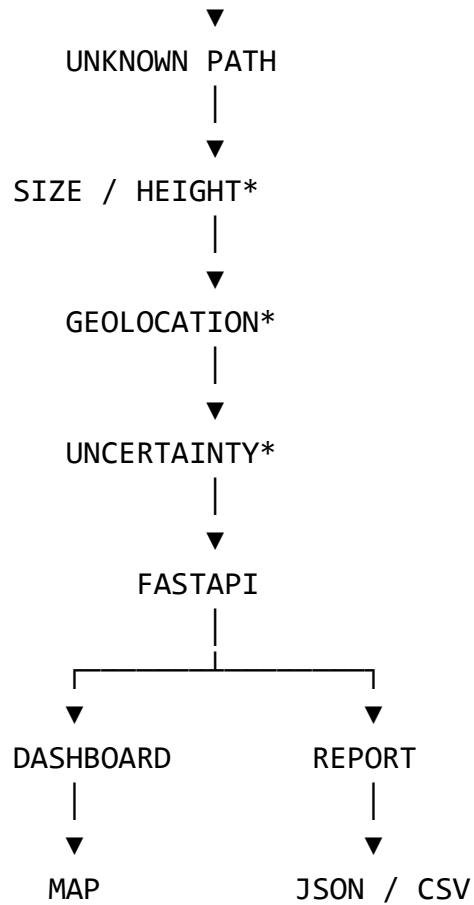
THE FINAL GOAL

By the end of **September 29**, the team should not merely have:

YOLO model
+
website

The intended result is:





The **original SIH-oriented plan** already calls for the combination of AI model, preprocessing/confidence/geotagging, dashboard, synthetic data, integration and deployment; the schedule above turns that long-form plan into your four actual development days while keeping the feature ownership and handoffs explicit.

This is now the **final operational stage** of the five-stage planning process: **Stage 1 = common product knowledge, Stage 2 = ownership, Stage 3 = features, Stage 4 = milestone definitions, Stage 5 = daily execution and handoffs.**

